Documentación técnica

Manual técnico · Arquitectura · Registro de decisiones — GS08 · Matrículas y Notas

Curso: Implementación de Soluciones para Plataformas Web · GS08 (ISP-08-2025)

Docente: Dr. Jack Harry Peralta Cáceres

Proyecto: GS08 · Matrículas y Notas — Proyecto Integrado (Trabajo Colaborativo)

Integrantes: Arteaga Rullier, Roberto Carlos (Scrum Master y DevOps) · Díaz Cárdenas, Jorge Luis (Backend) · Inocencio Gargate, Dalcir (Frontend y Documentación)

Repositorio: https://github.com/rarteagar/gs08-matriculas

Aplicación: https://gs08-matriculas.mlarca.com

Fecha: 19/10/2026

Manual técnico

Entregable: TD-04 de `docs/backlog-sprints.md` · **Redacta:** @documentador · **Fecha:** 21/09/2026

Rama: `main` @ `e051d43` · **Repo:** `D:\dev\equipo\gs08-matriculas` · **Máquina:** ASUS M1502YA (Windows 11 + Docker Desktop)

Regla de este manual: cada afirmación cita el archivo o el comando del que sale, y **nada que no exista se documenta como existente**. Lo que todavía no está implementado va marcado como *«pendiente T\<xx\>»*.

Actualizado el 13/10/2026: el frontend se migró de Vue 3 a **React 19 + Vite + Tailwind** y ahora consume las **30 operaciones** del contrato (D-27); al API se le añadieron **MongoDB, Redis, GraphQL y logging centralizado** (D-28) y la autenticación pasó a **OAuth2 con scopes por rol** (D-29), con lo que el contrato queda en **20 rutas / 35 operaciones** (v0.3.0). El API real, el SPA y el cluster ya estaban entregados el 22/09/2026.

Índice

1. Estado real del sistema (#1-estado-real-del-sistema)
2. Contrato de la API (#2-contrato-de-la-api)
3. Versiones del stack y su fuente (#3-versiones-del-stack-y-su-fuente)
4. Contenedores, red y variables (#4-contenedores-red-y-variables)
5. Base de datos: inicialización y verificación (#5-base-de-datos-inicialización-y-verificación)
6. Integración continua y entrega (#6-integración-continua-y-entrega)
7. Kubernetes (#7-kubernetes)
8. Observabilidad (#8-observabilidad)
9. Problemas conocidos (#9-problemas-conocidos)
10. Verificación rápida (5 comandos) (#10-verificación-rápida-5-comandos)
11. Archivos citados en este manual (#11-archivos-citados-en-este-manual)

1. Estado real del sistema

Actualizado el 22/09/2026 (ronda de cierre): el API real, el SPA de 4 vistas y el cluster están entregados, y el recorrido de extremo a extremo sale verde por el proxy (**39 OK, 0 fallas**).

Capa	Estado	Evidencia
API FastAPI real	✅ **existe, corre y está probado de punta a punta: 20 rutas / 35 operaciones** (`/openapi.json` v0.3.0), recorrido completo verde por el proxy	`bash scripts/smoke_api.sh --e2e` → **39 OK, 0 fallas, 0 bloqueadas, exit 0** (`docs/qa/evidencia/smoke-api-20260922-000358.txt`); capa nueva: **121 pruebas, 90 % de cobertura**
Autenticación, panel, estudiantes, cursos, matrículas	✅ **funcionando en el compose real**: POST `/api/v1/auth/login` → 200 + token; `/api/v1/dashboard` → **12 / 7 / 23 / 1**; `?q=45123456` → 1 fila con acentos correctos; `?q=huaman` = `?q=Huamán` = `?q=HUAMAN` → 1/1/1	salidas reales en §2.1
Notas y boleta	✅ **funcionando:** en el recorrido E2E del smoke, 3 notas → boleta	`docs/qa/evidencia/smoke-api-20260922-000358.txt`

	con **16.00** y promedio ponderado	
`POST /api/v1/matriculas` y `POST /api/v1/usuarios`	✅ **cerrados:** los dos `500` por `AmbiguousParameter` (BUG-08/BUG-09) quedaron sin reproducir en la corrida verde de 39/39	`docs/qa/evidencia/api-500-ambiguousparameter.txt` (el bug) · smoke 39/39 (el cierre)
SPA React	✅ **existe: 6 vistas** (login, panel, estudiantes, cursos, matrículas, usuarios) más los diálogos de boleta y notas, servidos por nginx en `:8080`	`curl -s -o /dev/null -w '%{http_code} %{content_type}' http://localhost:8080/` → `200 text/html`
Contenedores, red, balanceo, failover	✅ **funcionando**	`docker compose ps` (6 contenedores `healthy`), `verificar_stack.py` → 9 OK exit 0
Esquema de datos + reglas en la base	✅ **funcionando**, y **los dos caminos coinciden** (D-14 cerrado)	`verificar_modelo.sql` → **54 OK** y `verificar_notas.sql` → **32 OK**; `bash db/verificacion/comparar_con_alembic.sh` → **exit 0**, esquema y datos idénticos entre `db/init/` y Alembic
Observabilidad (Prometheus + Grafana)	✅ **funcionando con el tráfico real** (paneles 8 y 9 sin el ruido del scrape, `d42fcfc`)	p95 **0,095 s**; las dos instancias con carga (1,6 / 2,4 req/s)
Kubernetes	✅ **desplegado en la máquina**: 3 pods `Running` y el Job de migración `Complete 1/1` (`83140fd`); **Ingress creado pero addon apagado** (anotado)	`kubectl get pods -n gs08` · `kubectl get job -n gs08`
CI/CD en GitHub	⛔ escrito, **nunca ejecutado**: el repo no tiene remoto; lo hace el dueño al final	`docs/devops/plan-contenedores-ci.md` §4.6

Estado de los criterios hoy (cifra de @qa, `docs/qa/plan-pruebas.md` §3): **29 criterios en verde** con corrida real · **6 pendientes** del guion manual de interfaz · `C-37` y `C-39` en su estado declarado (Kubernetes desplegado sin Ingress activo; CI sin remoto público). Los criterios que estaban incumplidos por BUG-08/BUG-09 quedaron verdes en la corrida de 39/39.

> ***El único `:8080` sirve la app real.*** *El arnés de humo con el stub ya no está en juego: el compose del repo levanta la API real (`"motor":"PostgreSQL 16.15","entorno":"local"`) y el SPA de Vite. Se puede distinguir en un comando: `curl -s http://localhost:8080/api/v1/health` → el campo `entorno` es del API real, y `curl -o /dev/null -w '%{http_code}' http://localhost:8080/health` →* ***404*** *(D-17, nginx ya no devuelve el `index.html` en esas rutas).*

2. Contrato de la API

2.1 Lo que existe HOY: el API real (35 operaciones)

Contrato real, enumerado desde su propio OpenAPI (**API del compose, por el proxy en `:8080`**):

```
$ curl -s http://localhost:8080/openapi.json | python -c "import sys,json;
d=json.load(sys.stdin); print(d['info']); [print(m.upper(), p) for p,v in d['paths'].items()
for m in v]"
{'title': 'GS08 · Matrículas y Notas', 'version': '0.1.0',
```

```
 'description': 'API del MVP: autenticación, panel, estudiantes, cursos, matrículas, usuarios,
notas y boleta.
                Esquema de datos: db/init/*.sql y la revisión 0001 de Alembic.'}
GET  /health                                    GET  /api/v1/health
POST /api/v1/auth/login                         GET  /api/v1/auth/yo
GET  /api/v1/estudiantes                        POST /api/v1/estudiantes
GET  /api/v1/estudiantes/{estudiante_id}        PUT  /api/v1/estudiantes/{estudiante_id}
DELETE /api/v1/estudiantes/{estudiante_id}      GET  /api/v1/estudiantes/{id}/boleta
GET  /api/v1/cursos                             POST /api/v1/cursos
GET  /api/v1/cursos/{curso_id}                  PUT  /api/v1/cursos/{curso_id}
DELETE /api/v1/cursos/{curso_id}
GET  /api/v1/matriculas                         POST /api/v1/matriculas
GET  /api/v1/matriculas/{matricula_id}          PUT  /api/v1/matriculas/{matricula_id}
DELETE /api/v1/matriculas/{matricula_id}
GET  /api/v1/matriculas/{matricula_id}/notas    POST /api/v1/matriculas/{matricula_id}/notas
PUT  /api/v1/notas/{nota_id}                    DELETE /api/v1/notas/{nota_id}
GET  /api/v1/usuarios                           POST /api/v1/usuarios
GET  /api/v1/usuarios/{usuario_id}              PUT  /api/v1/usuarios/{usuario_id}
DELETE /api/v1/usuarios/{usuario_id}            GET  /api/v1/dashboard
```

Respuestas reales por el proxy, corridas por @documentador el 22/09/2026 contra el stack del compose:

```
$ curl -s http://localhost:8080/api/v1/health
{"status":"ok","motor":"PostgreSQL 16.15","instancia":"5e3d89c30030","entorno":"local"}

$ curl -X POST http://localhost:8080/api/v1/auth/login -H 'Content-Type: application/json' \
       -d '{"usuario":"admin","password":"Admin123!"}'
-> HTTP 200 + access_token (JWT de 184 caracteres)

$ curl -s -H "Authorization: Bearer $TOKEN" http://localhost:8080/api/v1/dashboard
{"estudiantes_activos":12,"cursos_activos":7,"matriculas_activas":23,"usuarios_activos":1,
 "top_cursos":[{"curso_id":1,"codigo":"C101","curso":"Matemática
Básica","creditos":4,"matriculas":5},
              {"curso_id":2,"codigo":"C102","curso":"Comunicación
Efectiva","creditos":3,"matriculas":4},
              {"curso_id":3,"codigo":"C201","curso":"Programación Web
III","creditos":4,"matriculas":4},
              {"curso_id":6,"codigo":"C204","curso":"Estadística
Aplicada","creditos":3,"matriculas":3},
              {"curso_id":5,"codigo":"C203","curso":"Inglés
Técnico","creditos":2,"matriculas":3}],
 "ultimos_estudiantes":[{"id":12,"codigo":"E20260012","dni":"56234567",
                         "nombres":"Fiorella Milagros","apellidos":"Cárdenas Ruiz", …}]}

$ curl -s -H "Authorization: Bearer $TOKEN"
"http://localhost:8080/api/v1/estudiantes?q=45123456"
{"total":1,"page":1,"page_size":25,"paginas":1,"estudiantes":[{"id":1,"codigo":"E20260001","dni
":"45123456",
 "nombres":"Carlos Alberto","apellidos":"Ramírez Torres","nombre_completo":"Ramírez Torres,
Carlos Alberto",
 "email":"cramirez@correo.pe","telefono":"987654321","fecha_nacimiento":"2005-03-14",
 "direccion":"Av. Los Laureles 245, Lima","estado":true,"creado_en":"2026-09-
22T04:04:11.429641Z"}]}

$ curl -s -H "Authorization: Bearer $TOKEN" "http://localhost:8080/api/v1/estudiantes?q=huaman"
-> total=1, «Quispe Huamán, María Fernanda»   (el buscador sin tildes encuentra el dato con
tilde)
```

Tres cosas que estas salidas demuestran: los **acentos viajan correctos en UTF-8** (`Ramírez`, `Huamán`, `Cárdenas`), el **KPI coincide con el seed** (12 / 7 / 23 / 1) y el **buscador sin tilde funciona** (`q=huaman` → 1, gracias a `unaccent`).

Todo lo de §2.1 sale del stack real del compose, por el proxy en `:8080`. *El stub de humo desapareció cuando el SPA real se compiló (el arnés de humo ya no participa), y el `:8010` de las corridas de la ronda anterior era una instancia de pruebas aparte, ya apagada. Cómo saber que es el API real y no un stub: `curl -s http://localhost:8080/api/v1/health` devuelve `"entorno":"local"` y un `instancia` que es el id del contenedor, y `curl -o /dev/null -w '%{http_code}' http://localhost:8080/health` responde **404** (D-17: nginx ya no devuelve el `index.html` en esas rutas).*

2.2 Contrato completo, con el estado real de cada fila

Fuente: `docs/analisis/modelo-datos.md` §4 (casos de uso CU-01…CU-15), `docs/alcance-mvp.md` §4 (criterios `C-01`…`C-39`) y `docs/devops/plan-contenedores-ci.md` §3 (contrato con @dev).

Estados (los de @qa, `docs/qa/plan-pruebas.md` §3): ✅ implementado y verificado con corrida · ☐ escrito sin verificación · 🔒 fuera de la demo por el alcance final (`docs/alcance-final.md`).

Método y ruta	Qué hace	Códigos	Criterios	Estado
`GET /health`	salud del API **con** verificación de PostgreSQL	200	C-34	✅ (por `:8000`; por el proxy falta TC-03)
`GET /api/v1/health`	lo mismo, para entrar por el proxy	200	C-34	✅
`GET /metrics`	métricas en formato Prometheus (instrumentator 8.1.0)	200	C-34	✅ directo en `:8000` (D-17)
`POST /api/v1/auth/login`	entra con **usuario o email** (`WHERE nombre_usuario = :u1 OR email = :u2`, igual que `login.php`)	200 + token · **401** clave mala o usuario inactivo	C-01, C-02	✅ verificado con corrida
`GET /api/v1/auth/yo`	datos del usuario de la sesión	200 · 401	—	☐
`GET /api/v1/dashboard`	4 KPIs + top 5 cursos + últimos 6 estudiantes, desde `v_matriculas_detalle`	200 (con seed: **12 / 7 / 23 / 1**)	C-05, C-06, C-07	✅ verificado con corrida
`GET /api/v1/estudiantes?q=&estado=&page=&page_size=`	listado paginado + buscador (código, DNI, nombres, apellidos)	200 · **422** si `page_size` es 0/200/abc	C-08, C-09, C-10	✅ q=huaman/Huamán/HUAMAN → **1/1/1** (con `unaccent`)
`POST /api/v1/estudiantes`	alta	**201** · **422** (DNI de 7 dígitos, repetidos, email	C-11	✅ verificado (422

		inválido). **Nunca 500**		con el campo)
PUT /api/v1/estudiantes/{id}	edición (incluye baja lógica con `estado=false`)	200 · 422 · 409 (auto-desactivarse)	C-11, C-12, C-04	✅ el 409 de la propia cuenta (RN-11)
DELETE /api/v1/estudiantes/{id}?confirmar=	borrado destructivo en dos pasos	**409** `{"detail":"…","matriculas":N}` sin `?confirmar` · **204** con `?confirmar=true`	C-30, C-31, C-32	✅ verificado
GET /api/v1/cursos?q=&estado=	listado y filtro de cursos	200	—	✅
POST·PUT·DELETE /api/v1/cursos/{id}	alta/edición/baja; `creditos` 1..10, `horas` 1..1000, default `creditos=3`	**422** (`creditos=0/11`, `horas=0`) · **409** (código repetido)	C-13, C-14, C-15	✅ los 422 y el 409 de código repetido
GET /api/v1/matriculas?q=&periodo=	listado con estudiante y curso (vista), filtro por texto y periodo	200 · **422** (periodo `2026-13`, `2026/02`)	C-18, C-21, C-24	✅
POST /api/v1/matriculas	crear matrícula	**201**; **409** «Ese estudiante ya está matriculado en ese curso para el periodo indicado»; **404** por FK	C-16, C-17, C-19	✅ **verificado en la corrida E2E** (el `500` de BUG-08 ya no se reproduce)
PUT /api/v1/matriculas/{id}	editar y **retirar** (`estado="retirado"`, baja lógica)	200 (el KPI baja exactamente **1** y la fila sigue en el historial)	C-20	✅ verificado (retirar y conservar notas)
DELETE /api/v1/matriculas/{id}	eliminar matrícula	204	—	☐
GET·POST /api/v1/usuarios	gestión de usuarios (**solo `admin`**)	**403** para `asistente` · **409** `nombre_usuario` repetido · **422** clave de 7 o rol inválido	C-22, C-23	✅ con la corrida de 39/39 (el `500` del POST de BUG-09 ya no se reproduce) · **fuera de la demo** según `docs/alcance-final.md`
POST /api/v1/matriculas/{id}/notas	registrar nota	**201** · **422** (`nota` 21/-1, `tipo="examen"`) · **409** duplicado **y** nota sobre matrícula `retirado` (RN-14)	C-24, C-25, C-26	✅ **funcionando** (verificado a mano por @qa)
PUT·DELETE /api/v1/notas/{id}	editar / eliminar nota	200 · 204	—	☐
GET /api/v1/estudiantes/{id}/boleta?periodo=	boleta con nota por curso y promedio ponderado por	200: `14/16/18` → `16.00`; curso sin notas → `nota: null` (no cuenta	C-27, C-28, C-29	✅ **funcionando**: estudiante 1 → C101 **16.00**,

	créditos	como 0)		**promedio ponderado 13.43** = (16×4 + 10×3) ÷ 7

2.3 Capa de rúbrica del API: MongoDB, Redis, GraphQL y logging (D-28)

El núcleo sigue siendo FastAPI + PostgreSQL 16; encima se añadieron cuatro capas para el ítem «Backend seguro y eficiente — APIs + BD híbridas» de la consigna. Todas **degradan solas**: si la pieza no está configurada o no responde, el API contesta igual (el CI solo trae PostgreSQL, y sigue verde). Si la variable está puesta pero la pieza no responde, **no se apaga para siempre**: se reintenta cada 30 s (`ENFRIAMIENTO_SEGUNDOS` en `app/mongo.py` y `app/cache.py`, D-31).

Pieza	Dónde vive	Qué hace	Cómo se comprueba
MongoDB (segunda base)	`app/mongo.py`, base `gs08_bitacora`, colecciones `bitacora` y `boletas_snapshot`	La **bitácora** guarda cada escritura (POST/PUT/DELETE de `/api/v1/...`) con usuario, rol, ruta, estado, demora y cuerpo **sin credenciales**; la boleta emitida se archiva como documento	`GET /api/v1/auditoria` (solo admin) y `GET /api/v1/estudiantes/{id}/boleta/archivada`
Redis (caché)	`app/cache.py`, clave `gs08:panel`, TTL 60 s	cachea el panel (3 consultas agregadas) y el middleware la **invalida** en cada escritura correcta	`docker exec gs08-redis redis-cli keys 'gs08:*'` / `ttl gs08:panel`
GraphQL	`app/graphql.py`, montado en `/graphql` (GraphiQL incluido)	Consultas `panel`, `estudiantes(q)`, `estudiante(id)`, `cursos(q)`, `matriculas(estudianteId, periodo)` y `boleta(estudianteId, periodo)`, con la **misma** fórmula de boleta que el REST (`app/boleta.py`)	`POST /graphql` con el token; sin token → **401**
Logging centralizado	`app/observabilidad.py`	Cada petición sale en **una línea JSON** con `pedido_id`, usuario, ruta, estado y `demora_ms`; la respuesta lleva la cabecera `X-Request-ID`; con `LOKI_URL` el mismo log se empuja a Loki y se ve en Grafana	`docker logs gs08-api` y Grafana → Explore → Loki

Rutas nuevas: el contrato pasa de 16 rutas / 30 operaciones a **19 rutas / 34 operaciones** (`/openapi.json`, v0.2.0): `GET /api/v1/auditoria`, `GET /api/v1/estudiantes/{id}/boleta/archivada` y `POST /graphql`.

Quién escribe la bitácora: el middleware, no cada endpoint. Así ninguna operación nueva queda sin auditar por olvido, y la credencial nunca se guarda: los campos que suenan a clave (`password`, `token`,

`secret`…) se sustituyen por `***` antes de tocar Mongo (`tests/test_observabilidad.py::test_el_login_no_guarda_la_contrasena_en_la_bitacora`).

Auditoría de la caché medida (13/10/2026): con 20 000 estudiantes y 20 000 matrículas, el panel pasa de

54,2 ms a 13,5 ms (4,0x); con el seed de 12 estudiantes la diferencia es de ~2 ms, porque las consultas agregadas ya eran baratas. La caché no es un adorno: se mide y se reporta lo que da.

2.4 Reglas transversales del contrato

Regla	Detalle	Fuente	
Formato de error	Cuerpo `{"detail": "…"}` con el mensaje **del backend** (el SPA lo muestra tal cual, sin diálogos nativos del navegador)	`docs/qa/plan-pruebas.md` §7, `docs/alcance-mvp.md` §4	
Nunca 500 por dato del usuario	Un DNI corto, un periodo imposible o un `page_size` absurdo son **422/409**, no 500	C-11, C-18, C-10	
Paginación	25 por página (`page`, `page_size`); fuera de rango → 422	D-07	
Buscador	`ILIKE` sobre código, DNI, nombres y apellidos; **`unaccent` obligatorio** para tildes	D-11 + **BUG-07**	
Periodo	`varchar(7)` con CHECK `^[0-9]{4}-(0[1-9]|1[0-2])$ (ej. 2026-02`)	`db/init/01-schema.sql`	
Notas	`tipo` ∈ {`practica`,`parcial`,`final`}, `numero ≥ 1`, nota 0-20; nota del curso = **media aritmética**; promedio del periodo = **ponderado por créditos**	D-03	
Booleans	`estado` es `true`/`false` en el JSON (en MySQL era `1`/`0`)	`docs/analisis/modelo-datos.md` §2	
Fechas	`timestamptz` en ISO-8601 con zona; la API responde en `America/Lima`	idem	
Salud y métricas	`/api/v1/health` por el proxy; `/metrics` **directo** en `:8000`; nginx **404** en `/health` y `/metrics`	D-17	

2.6 Autenticación y permisos: OAuth2 con scopes (D-29)

El API es su propio servidor de autorización: **OAuth2 password grant** (RFC 6749 §4.3) en `POST /api/v1/auth/token`, con **scopes** por rol. No hay Keycloak ni proveedor externo: es el mismo proceso FastAPI, la misma imagen y ningún contenedor nuevo.

Pieza	Cómo funciona
`POST /api/v1/auth/token`	Form-urlencoded: `grant_type=password`, `username`, `password` y `scope` (opcional: sin él se firman todos los del rol). `client_id`/`client_secret` son opcionales (el SPA es cliente público) y se validan si llegan
Respuesta	`{access_token, token_type: "bearer", expires_in, scope}` con la forma del RFC 6749 §5.1
`POST /api/v1/auth/login`	Alias en JSON para el SPA (no cambia nada en el frontend). El token que emite **ya lleva los scopes** del rol
Token	JWT HS256 con `sub`, `usuario`, `rol`, **`scp`** (lista de scopes), `iat`, `exp`
Autorización	Cada endpoint exige su scope: `Depends(requiere_scope("matriculas:escribir"))`. Si al token le falta → **403** `{"error":"insufficient_scope"}` + `WWW-Authenticate: Bearer error="insufficient_scope", scope="..."`
Errores del token	`invalid_client` (401), `invalid_grant` (400), `invalid_scope` (400), `unsupported_grant_type` (400), todos con `error` / `error_description`
Swagger	Botón **Authorize** con el flujo OAuth2 y el candado por endpoint; `/auth/yo` exige `perfil:leer`

Los 16 scopes y quién los tiene:

Scope	admin	asistente
`perfil:leer`, `panel:leer`, `estudiantes:leer`, `cursos:leer`, `matriculas:leer`, `notas:leer`, `boleta:leer`	✅	✅
`estudiantes:escribir`, `cursos:escribir`, `matriculas:escribir`, `notas:escribir`	✅	✅
`estudiantes:eliminar`, `cursos:eliminar`, `matriculas:eliminar`	✅	❌
`usuarios:admin`, `auditoria:leer`	✅	❌

Ese reparto reproduce exactamente el comportamiento anterior (cuando existía la dependencia `solo_admin`): el rol ya no se consulta en cada endpoint, se consulta **una vez** al emitir el token, que

sale con los permisos que le corresponden. La bitácora guarda cada petición de token con la contraseña y el secreto en ***.

```
# token con todos los scopes del rol
curl -s localhost:8000/api/v1/auth/token -d grant_type=password -d username=admin -d password=…

# token limitado: sirve para leer estudiantes y para nada más
curl -s localhost:8000/api/v1/auth/token -d grant_type=password -d username=admin -d password=…
-d "scope=estudiantes:leer"
```

2.7 Requisitos que la infraestructura le impone al código (contrato con @dev)

Fuente: `docs/devops/plan-contenedores-ci.md` §3. Si el código no cumple esto, **el stack no arranca**:

Punto	Requisito
Punto de entrada	`backend/app/main.py`, arrancado con `uvicorn app.main:app --host 0.0.0.0 --port 8000` (el CMD del Dockerfile ya lo trae, con `--proxy-headers`)
Healthchecks usados por Docker y k8s	`GET /health` → **200** verificando PostgreSQL
Métricas	`GET /metrics` con `prometheus-fastapi-instrumentator==8.1.0`
Variables	`DATABASE_URL` (la inyecta compose), `SECRET_KEY`, `ENVIRONMENT`, `LOG_LEVEL`, `ACCESS_TOKEN_EXPIRE_MINUTES`
Dependencias	`backend/requirements.txt` con versiones **fijadas** (hoy **no existe**: `docker compose build api` falla en `COPY requirements.txt`)
Migraciones	Alembic en `backend/alembic/`; en el cluster corre el Job `gs08-migraciones` con `alembic upgrade head`
Frontend	`frontend/package.json` con `build` que genere `dist/` y **`package-lock.json` commiteado** (`npm ci` lo exige)

3. Versiones del stack y su fuente

Herramientas de la máquina (verificadas por @documentador el 21/09/2026; ampliación en `docs/devops/COMANDOS.md` §0):

Herramienta	Versión	Comando			
Docker Engine	`29.8.0, build 88096ef`	`docker --version`			
Docker Compose	`v5.5.1`	`docker compose version`			
Docker daemon	`29.8.0 \	linux \	overlayfs \	Mem=12 GB`	`docker info --format '…'`
kubectl / kustomize	`v1.36.1 / v5.8.1`	`kubectl version --client`			
Python	`3.11.16`	`python --version`			

Node	v26.1.0	node --version			

Imágenes (todas fijadas; `scripts/validar_infra.py` falla si aparece un `:latest`):

Imagen	Versión	Se declara en
PostgreSQL	`postgres:16.15-alpine3.24`	`docker-compose.yml`, `k8s/10-postgres.yaml`
Python del API	`python:3.12.14-slim-bookworm`	`backend/Dockerfile` (2 etapas: `deps` → `runtime`)
Node del SPA	`node:22.23.2-alpine`	`frontend/Dockerfile`
nginx	`nginx:1.31.6-alpine`	`frontend/Dockerfile`
Prometheus	`prom/prometheus:v3.13.3`	`docker-compose.yml`
Grafana	`grafana/grafana:12.4.11`	`docker-compose.yml`

Librerías del API — versiones **contratadas y verificadas contra PyPI el 21/09/2026**, todavía **no instaladas** (`backend/requirements.txt` es T1.1). Fuente: `docs/devops/plan-contenedores-ci.md` §3.

```
fastapi==0.141.1 · uvicorn==0.53.0 · SQLAlchemy==2.0.54 · psycopg[binary]==3.3.6
prometheus-fastapi-instrumentator==8.1.0 · pydantic-settings==2.15.0 · alembic==1.20.0
```

Verificación de contraseñas: `bcrypt==4.2.1` (PyPI) — es la librería con la que se comprobó que el hash `$2y$` del seed corresponde a `Admin123!` (`docs/analisis/evidencia/hash-legacy-python-bcrypt.txt`).

Frontend: React 19 + Vite + Tailwind, **sin librería de estado ni de componentes** (D-16, ajustada por D-27).

4. Contenedores, red y variables

4.1 Servicios del compose

`docker compose config --services` → `db`, `mongo`, `redis`, `api`, `api-b`, `web` (+ `prometheus`, `grafana`, `loki` con `--profile obs`).

Servicio	Contenedor	Imagen	Puerto al host	Notas
db	gs08-db	postgres:16.15-alpine3.24	5432	volumen pgdata; POSTGRES_INITDB_ARGS: --encoding=UTF8 --locale=C; healthcheck pg_isready
mongo	gs08-mongo	mongo:8.0	27017	volumen mongodata; base gs08_bitacora; healthcheck mongosh db.adminCommand('ping'); segunda base de la rúbrica (bitácora y boletas archivadas)
redis	gs08-redis	redis:8-alpine	6379	volumen redisdata; --appendonly yes --

				maxmemory 128mb --maxmemory-policy allkeys-lru; healthcheck redis-cli ping
api	gs08-api	gs08-api:local (build backend/)	8000	depende de db, mongo y redis en estado healthy
api-b	gs08-api-b	la misma imagen	—	segunda instancia para el balanceo; **sin puerto al host** a propósito
web	gs08-web	gs08-web:local (build frontend/)	8080	nginx; API_UPSTREAM con las dos instancias
prometheus	gs08-prometheus	prom/prometheus:v3.13.3	9090	perfil obs; retención 15 d; --web.enable-lifecycle
grafana	gs08-grafana	grafana/grafana:12.4.11	3000	perfil obs; dashboard y datasources (Prometheus y Loki) provisionados
loki	gs08-loki	grafana/loki:3.5.2	3100	perfil obs; volumen lokidata; retención 14 d; recibe los logs JSON del API

Red: una sola red bridge, gs08-net (name: gs08-net, driver bridge). Los contenedores se resuelven por **nombre de servicio** (api, api-b, db, mongo, redis), que es lo que usa el upstream de nginx.

Volúmenes con nombre: pgdata, mongodata, redisdata (datos), prometheus_data (15 d), grafana_data (paneles y usuarios), lokidata (logs).

Logs acotados: driver json-file con max-size: 10m y max-file: 3 en todos los servicios.

4.2 Variables de entorno

Plantilla: .env.example (compose funciona **sin** .env, usando esos valores por defecto).

Variable	Default de desarrollo	Para qué
POSTGRES_DB / POSTGRES_USER / POSTGRES_PASSWORD	gs08_matriculas / gs08 / gs08_dev_pwd	base de datos
DB_PORT / API_PORT / WEB_PORT	5432 / 8000 / 8080	puertos publicados
ENVIRONMENT / LOG_LEVEL	local / info	modo y verbosidad del API
SECRET_KEY	cambiar-esta-clave-en-produccion	firma del token
ACCESS_TOKEN_EXPIRE_MINUTES	60	vigencia de la sesión
VITE_API_BASE_URL	/api/v1	base de la API embebida en el bundle (mismo origen: nginx proxea)
MONGO_URL / MONGO_BD	mongodb://mongo:27017 / gs08_bitacora	segunda base documental (bitácora y boletas archivadas). **Vacío = apagada**: el API responde igual
REDIS_URL / CACHE_TTL_SEGUNDOS	redis://redis:6379/0 / 60	caché distribuida del panel. **Vacío = apagada**, sin caché

LOKI_URL / SERVICIO	(vacío) / gs08-api	si tiene valor, además de la consola los logs JSON se empujan a Loki; SERVICIO es la etiqueta con la que se filtran
MONGO_PORT / REDIS_PORT / LOKI_PORT	27017 / 6379 / 3100	puertos publicados de los servicios nuevos
GRAFANA_ADMIN_USER / GRAFANA_ADMIN_PASSWORD	admin / gs08_grafana	acceso a Grafana
PROMETHEUS_PORT / GRAFANA_PORT	9090 / 3000	observabilidad
TZ	America/Lima	zona horaria de todos los contenedores
API_UPSTREAM	(se declara en docker-compose.yml y el ConfigMap de k8s)	directiva server que sustituye envsubst en la plantilla de nginx

Son credenciales de desarrollo, declaradas como tales: antes de cualquier entrega se cambian POSTGRES_PASSWORD y SECRET_KEY (docs/devops/plan-contenedores-ci.md §6).

4.3 nginx: SPA + proxy + balanceo

frontend/nginx/default.conf.template → el entrypoint de la imagen aplica envsubst con API_UPSTREAM y genera /etc/nginx/conf.d/default.conf. Lo importante del archivo:

Bloque	Qué hace
upstream gs08_api	${API_UPSTREAM} + zone gs08_api 64k; (sin zone, cada worker lleva su contador y el reparto sale desparejo) + keepalive 16
listen 8080	nginx corre como **usuario `nginx`** (no puede escuchar en 80)
location /healthz	responde ok — **healthcheck del contenedor** (lo usa Docker y k8s)
location /api/	proxy_pass http://gs08_api, proxy_next_upstream error timeout (reintenta solo en error/timeout, **no** en 5xx para no repetir escrituras), X-Upstream-Addr
location /assets/	caché de 30 d (immutable)
location /	try_files $uri $uri/ /index.html — el SPA no se rompe al refrescar una ruta profunda

Tres comprobaciones sobre este archivo (todas con su salida en §9): balanceo alternando IPs, 404 en /health y /metrics (TC-03) y las 4 cabeceras también en /api/ (TC-02).

4.4 Dockerfiles

Archivo	Detalles
backend/Dockerfile	2 etapas: venv en deps → runtime sin compiladores;

	usuario app (uid/gid 10001); HEALTHCHECK con `curl -fsS /health`; CMD con `--workers 2 --proxy-headers --forwarded-allow-ips '*'`
`frontend/Dockerfile`	etapa `build` (Node 22 → `npm ci && npm run build`) → etapa `runtime` (nginx alpine); `USER nginx`, escucha 8080, directorios escribibles; HEALTHCHECK con `wget /healthz`; `ARG VITE_API_BASE_URL` inyectado en el bundle

Ambos verificados por `scripts/validar_infra.py`: exige `USER` explícito, `HEALTHCHECK` y `.dockerignore`.

5. Base de datos: inicialización y verificación

5.1 Cómo se inicializa

`docker-compose.yml` monta `./db/init` en `/docker-entrypoint-initdb.d:ro`. El entrypoint de `postgres` ejecuta los `*.sql` **en orden alfabético y una sola vez**, al crear el volumen: `01-schema.sql` (4 tablas, 16 CHECK, 6 UNIQUE, 2 FK, 7 índices) → `02-view.sql` (`v_matriculas_detalle`) → `03-seed.sql` (1 admin, 12 estudiantes, 7 cursos, 24 matrículas). Contrato de la carpeta: `db/init/README.md`.

> ***Consecuencia operativa:*** *si se cambia un `.sql`,* ***no*** *se aplica solo. Hay que recrear el volumen: `docker compose down -v && docker compose up -d db`. Es* ***destructivo*** *y por eso se ejecuta solo con aprobación explícita.*

5.2 Cómo se verifica (comando y salida real)

```
$ bash db/verificacion/ejecutar_verificacion.sh
verificar_modelo.sql      exit=0   comprobaciones OK=49   fallas=0
verificar_hash_admin.sql  exit=0   comprobaciones OK=2
```

Levanta un PostgreSQL desechable (**puerto 55432**, contenedor `gs08-analista-verif`) con el mismo usuario y la misma base que el compose, deja que el entrypoint ejecute `01/02/03` y corre las cuatro verificaciones. Las 49 comprobaciones incluyen **15 pruebas negativas** (se exige que la base devuelva SQLSTATE 23505/23514/23503), 4 dentro de una transacción (cascada en ambos sentidos y las dos secuencias IDENTITY) y 3 después del `ROLLBACK` (datos intactos). Detalle completo: `docs/analisis/modelo-datos.md` §5.

5.3 Las reglas viven en la base, no solo en el código (verificación cruzada)

`docs/devops/evidencia-sprint0.md` §7 — imagen del API contra el esquema, sin tocar datos:

```
tablas visibles para el API: ['cursos', 'estudiantes', 'matriculas', 'usuarios']
vistas visibles para el API: ['v_matriculas_detalle']
filas: cursos 7 | estudiantes 12 | matriculas 24 | usuarios 1   (acentos correctos en UTF-8)
reglas rechazadas por la base (correcto):
  matrícula duplicada  -> uq_matriculas_estudiante_curso_periodo
  dni de 3 dígitos     -> ck_estudiantes_dni
  créditos=99          -> ck_cursos_creditos
  periodo '2026/02'    -> ck_matriculas_periodo
```

5.4 Dos rutas del esquema y una sola verdad (D-14)

Hoy hay **dos DDL del mismo esquema**: los scripts `psql` de `db/init/` (que usa el compose al crear el volumen) y la migración de Alembic (que aplica el Job `gs08-migraciones` en el cluster). La decisión **D-14** es que la **revisión inicial de Alembic se genere desde `db/init/`**, para que no haya dos verdades.

Estado: pendiente — es T1.1. Cualquier cambio de esquema posterior (por ejemplo el `CREATE EXTENSION unaccent` de BUG-07) debe ir **en los dos lados** o la decisión se rompe.

5.5 Datos del seed (los números que no se tocan)

Dato	Valor	Fuente
Estudiantes / cursos / matrículas	**12 / 7 / 24** (23 activas + 1 retirada)	`docs/analisis/datos-seed.md` (medido con `psql`)
La única retirada	matrícula **13**: estudiante 7 (Castillo Neyra, Jorge Luis), curso 4 (C202 Base de Datos II), `2026-02`	idem
Único periodo	`2026-02` en las 24	idem
Login de demo	`admin` / `Admin123!` · `admin@horizonte.edu.pe`	idem
Borrado con matrículas	curso 5 → **3** · estudiante 2 → **2**	idem

6. Integración continua y entrega

6.1 CI (`.github/workflows/ci.yml`) — se dispara en `pull_request` y `push` a `main`, y a mano

Job	Qué corre	Se salta si…
`infra`	`python3 scripts/validar_infra.py`, `docker compose config -q`, `hadolint` en los dos Dockerfiles	**nunca**: es el que da verde en el Sprint 0
`backend`	`ruff check` + `ruff format --check` + `pytest --cov=app --cov-fail-under=60` contra PostgreSQL 16 como *service*	no existe `backend/requirements.txt` → **hoy: `skipped`**
`frontend`	`npm ci`, `eslint`, `vitest`, `npm run build`, artefacto `frontend-dist`	no existe `frontend/package.json` → **hoy: `skipped`**
`imagenes`	`docker compose build` + `up -d` + healthchecks + `trivy` (informativo) + `down -v`	—
`ci-ok`	puerta única para la protección de rama	—

Estado real: el CI nunca corrió en GitHub (no hay remoto). Lo que sí se corre en local es su equivalente:

```
$ python scripts/validar_infra.py
21 comprobaciones OK, 0 fallas
$ docker compose config -q      # OK, sin salida
```

6.2 CD (`.github/workflows/cd.yml`) — tras un CI verde en `main`, o a mano

Job	Qué hace	Guarda
`publicar`	buildx + push a **GHCR** con tag sha y	`permissions: packages: write`

	`main`, con **SBOM y provenance**	
desplegar	`kubectl apply -k k8s/`, set `image` con el tag nuevo, `rollout status`, humo contra el Ingress, y `rollout undo` si falla	**solo si `vars.DEPLOY_ENABLED == 'true'`** (hoy no existe: se salta y el pipeline no se rompe)

Credenciales que faltan para que el despliegue funcione: secret `KUBE_CONFIG_B64` y variable `DEPLOY_ENABLED=true` — ninguna existe todavía (A-02 / T4.1).

Acciones de GitHub usadas (fijadas por versión): `checkout v7.0.1`, `setup-python v7.0.0`, `setup-node v7.0.0`, `setup-buildx-action v4.4.1`, `build-push-action v7.4.0`, `login-action v4.6.0`, `hadolint-action v3.5.0`, `upload-artifact v7.0.1`, `trivy-action v0.36.0`.

7. Kubernetes

7.1 Estado: manifiestos listos, cluster NO desplegado en esta máquina

> ***Nota (13/10/2026):*** *los manifiestos de `k8s/` describen el stack **anterior** a D-28: no incluyen MongoDB, Redis ni Loki. El despliegue que se usa hoy es el compose (local y Coolify, que ya los trae). Añadirlos al cluster es trabajo pendiente y depende de A-02 (instalación de minikube).*

```
$ kubectl config current-context
error: current-context is not set
$ which minikube kind k3d k3s
ninguno instalado   (solo el cliente kubectl v1.36.1)
$ kubectl kustomize k8s/          # validación offline (la única posible sin cluster)
11 objetos
```

`kubectl apply --dry-run=client` **no** sirve sin cluster (`failed to download openapi`): la validación offline es `kubectl kustomize k8s/`. **Instalar minikube es software nuevo en la máquina: espera la autorización del dueño (A-02)**, por eso el criterio `C-37` figura como *bloqueado* y no se escribe `kubectl get pods` hasta que exista un cluster.

7.2 Los 11 objetos

Kind	Nombre	Detalle que importa
Namespace	`gs08`	todo vive acá
ConfigMap	`gs08-config`	`DB_HOST=gs08-db`, `ENVIRONMENT=k8s`, `API_UPSTREAM="server gs08-api:8000 …"`
Service	`gs08-api`	ClusterIP `:8000`
Service	`gs08-db`	ClusterIP `:5432` (**no** se publica al host)
Service	`gs08-web`	ClusterIP `:80` → contenedor `8080`
PersistentVolumeClaim	`gs08-pgdata`	2Gi, `ReadWriteOnce`
Deployment	`gs08-api`	2 réplicas, `RollingUpdate` `maxSurge 1 / maxUnavailable 0`, `runAsNonRoot` uid 10001, `capabilities: drop ALL`, probes `/health`, resources 100m/256Mi →

		1/512Mi
Deployment	`gs08-db`	1 réplica, `strategy: Recreate` (el volumen RWO no se monta dos veces)
Deployment	`gs08-web`	2 réplicas, probes `/healthz`
Job	`gs08-migraciones`	`alembic upgrade head`, `backoffLimit 3`, `ttlSecondsAfterFinished 600`, con initContainer que espera a PostgreSQL
Ingress	`gs08`	`ingressClassName: nginx`; `gs08.local/` → web y `gs08.local/api` → api; `proxy-body-size 8m`, `proxy-read-timeout 60`

El **Secret** no está en la lista a propósito: `k8s/02-secret.example.yaml` es la plantilla y el real se crea por comando (está en `.gitignore`).

7.3 Cómo se levanta (cuando A-02 se apruebe)

Runbook completo: `docs/devops/COMANDOS.md` §5. En resumen:

```
minikube start --cpus 4 --memory 6g
minikube addons enable ingress
eval $(minikube docker-env)          # o: minikube image load gs08-api:local gs08-web:local
docker compose build
kubectl -n gs08 create secret generic gs08-secrets --from-literal=…
kubectl apply -k k8s/
kubectl -n gs08 get pods,svc,ingress
echo "$(minikube ip)  gs08.local" >> /etc/hosts    # en Windows:
C:\Windows\System32\drivers\etc\hosts
curl -H "Host: gs08.local" http://$(minikube ip)/api/v1/health
```

Coherencia importante (D-14): en el cluster el DDL inicial lo aplica **Alembic** (Job `gs08-migraciones`), **no** `db/init/` — que solo corre con el volumen nuevo del compose.

8. Observabilidad

Detalle de cada panel y de los diagramas: `docs/arquitectura.md` §5. Lo esencial para operar:

Qué	Cómo	Salida real
Levantar la observabilidad	`docker compose --profile obs up -d`	6 contenedores en total
Targets	http://localhost:9090/targets	`gs08-api/instancia=api → up`, `instancia=api-b → up`, `prometheus → up`
Recargar la config de Prometheus	`curl -X POST http://localhost:9090/-/reload`	`HTTP 200`
Grafana	http://localhost:3000 con el dashboard `gs08-matriculas` ya provisionado	9 paneles con datos

Métricas crudas	http://localhost:8000/metrics (directo, D-17)	`# HELP http_requests_total …` (~10 KB)

Lo que NO se mide: PostgreSQL y nginx no exponen métricas Prometheus sin un exporter propio; el job de nginx que existía se eliminó porque fallaba (`ok\n` no es formato Prometheus). Agregarlos es Sprint 1 si el tiempo alcanza (`docs/devops/plan-contenedores-ci.md` §5). **Tampoco existe** la métrica `http_requests_inprogress` en el instrumentator 8.1.0: el panel que la usaba se reemplazó por «Errores 4xx (%)».

9. Problemas conocidos

> ***Aviso de vigencia (22/09/2026):*** *los bugs* ***05, 06, 07, 08 y 09*** *que se detallan abajo están* ***cerrados*** *(TC-02, TC-03,* `unaccent` *en los dos caminos y la corrida E2E de 39/39): sus bloques quedan como registro de lo que se encontró, con la salida de esa ronda. Lo que sigue vigente como problema son los apartados 9.8 a 9.11 y el primero de la lista del README (§8).*

Cada uno con **cómo se comprueba**. Los dos primeros están reproducidos hoy mismo, tal como siguen.

9.1 BUG-05 · `/health` y `/metrics` por el proxy devuelven el `index.html` (arreglo: TC-03)

```
$ curl -s -o /dev/null -w 'proxy 8080 /health   -> %{http_code} %{content_type}
%{size_download} bytes\n' http://localhost:8080/health
proxy 8080 /health   -> 200 text/html 371 bytes
$ curl -s -o /dev/null -w 'proxy 8080 /metrics  -> %{http_code} %{content_type}
%{size_download} bytes\n' http://localhost:8080/metrics
proxy 8080 /metrics  -> 200 text/html 371 bytes
$ curl -s -o /dev/null -w 'directo 8000 /metrics-> %{http_code} %{content_type}
%{size_download} bytes\n' http://localhost:8000/metrics
directo 8000 /metrics-> 200 text/plain; version=1.0.0; charset=utf-8 9952 bytes
```

Lectura: nginx solo proxea `/api/`; todo lo demás cae en el `try_files … /index.html`. Un chequeo de monitoreo que mire solo el código de estado da por bueno un `/metrics` sin métricas. **Decisión D-17:** `/metrics` se lee directo del API y nginx responde **404** en `/health` y `/metrics` (lo aplica @devops en TC-03 y lo verifica @qa). Detalle: `docs/qa/reporte-bugs.md` BUG-05.

9.2 BUG-06 · Las cabeceras de seguridad no llegan a `/api/` (arreglo: TC-02)

```
$ curl -s -o /dev/null -D - http://localhost:8080/healthz | grep -iE 'server|x-content|x-
frame|referrer'
Server: nginx
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Referrer-Policy: strict-origin-when-cross-origin

$ curl -s -o /dev/null -D - http://localhost:8080/api/v1/health | grep -iE 'server|x-content|x-
frame|referrer|x-upstream'
Server: nginx
X-Upstream-Addr: 172.18.0.5:8000
```

Causa: en nginx, una `location` que declara su propio `add_header` **no hereda** los del `server`; la `location /api/` declara `X-Upstream-Addr` y por eso pierde los tres. **Arreglo:** repetir los `add_header` dentro de la location (o `include` de un archivo común). Detalle: `docs/qa/reporte-bugs.md` BUG-06.

9.3 BUG-07 · Sin `unaccent` el buscador no encuentra tildes — CERRADO en los dos caminos

Consulta	Antes (sin unaccent)	Con unaccent

?q=huaman	**0 filas**	1
?q=Huamán	1	1
?q=HUAMAN	**0 filas**	1

`ILIKE` ignora mayúsculas pero **no** tildes; el dato del seed es `Quispe Huamán`. **Cerrado:** la extensión está en `db/init/01-schema.sql` **y** en la revisión inicial de Alembic (`backend/alembic/versions/0001_esquema_inicial.py:39` → `CREATE EXTENSION IF NOT EXISTS unaccent`), y `bash db/verificacion/comparar_con_alembic.sh` da **exit 0** con esquema y datos idénticos entre los dos caminos: **D-14 cerrado**. El API responde `1/1/1` (§2.1).

9.3-bis BUG-08 · `POST /api/v1/matriculas` respondía 500 — CERRADO

Fue el peor de los abiertos: una matrícula nueva y una duplicada devolvían las dos `500`, así que no se podía matricular por la API y el recorrido de §3 se cortaba en el paso 6 (afectaba `C-16`, `C-17` y `C-19`). La causa medida fue `psycopg.errors.AmbiguousParameter: could not determine data type of parameter $4` en `_verificar_duplicado()` (`app/routers/matriculas.py:129`): con `propio=None`, ni el `IS NULL` ni el `<>` le daban tipo al parámetro, y fallaba solo en el `INSERT` (en el `PUT` el valor llegaba).

Cerrado con corrida verde: `bash scripts/smoke_api.sh --e2e` → **39 OK, 0 fallas**, con el alta de matrícula y el `409` del duplicado dentro del recorrido (`docs/qa/evidencia/smoke-api-20260922-000358.txt`). Evidencia del bug original: `docs/qa/evidencia/api-500-ambiguousparameter.txt`.

9.3-ter BUG-09 · `POST /api/v1/usuarios` respondía 500 — CERRADO

El mismo patrón (`parameter $3`), que bloqueaba el `409` de `C-22` y el `403` de `C-23`. **Cerrado:** sin `500` en la corrida de 39/39. **Nota de alcance:** el módulo de usuarios quedó **fuera de la demo** por decisión del dueño (`docs/alcance-final.md`), así que no se muestra aunque funcione.

9.4 Quirk `$2y$` de pgcrypto: el hash del seed no se valida desde SQL (no es un bug)

`db/verificacion/diagnostico_hash_legacy.sql` demuestra que pgcrypto calcula un digest distinto con `$2y$` y solo coincide normalizando a `$2a$` (`verifica_2y = f`, `verifica_2a = t`). **No se toca el hash** (D-12): la verificación la hace la API con `bcrypt`. Si alguien intenta «comprobar el hash desde SQL», va a creer que el hash está mal: no lo está.

9.5 `LIKE` → `ILIKE` (`D-11`): el buscador del legacy no se puede copiar

Medido sobre 100.000 filas: `LIKE '%quispe%'` → **0 filas**, `ILIKE '%quispe%'` → 2. Un `LIKE` copiado del legacy deja el buscador **mudo y sin aviso**. El índice GIN trigram (112 ms → 0,09 ms) es un «si algún día crece», no parte del MVP. Evidencia: `docs/analisis/evidencia/busqueda-rendimiento.txt`.

9.6 Ids con huecos (`D-13`): no son un bug

`nextval` no se revierte con `ROLLBACK` y un INSERT rechazado por un CHECK **consume** el id (tras 3 INSERT fallidos, el siguiente curso válido fue **11**). El seed cierra con `setval`. **Nadie «arregla» las secuencias a mano.**

9.7 El Ingress de minikube está creado pero el addon de ingress quedó apagado

```
$ kubectl get pods -n gs08
gs08-api-6768c78484-pddz4   1/1   Running     0   4m17s
gs08-db-5746899898-7nxf7    1/1   Running     0   4m17s
gs08-migraciones-8hx2g      0/1   Completed   0   4m17s
gs08-web-bddfcdcf6-wclh5    1/1   Running     0   4m17s
```

```
$ kubectl get job -n gs08
gs08-migraciones   Complete   1/1   39s   4m17s
```

No es un fallo del despliegue: los 3 pods están arriba y el Job aplicó el esquema; lo que falta es el acceso por el host `gs08.local`. **Solución cuando se quiera entrar por ahí:** `minikube addons enable ingress`. Mientras tanto, el cluster se verifica por `kubectl` (`wget /healthz` → `ok` y `curl /api/v1/health` → **200** con `entorno: k8s` dentro del pod).

9.8 Los scripts de `db/init/` corren una sola vez

Si se cambia un `.sql`, hay que recrear el volumen (`docker compose down -v`, **destructivo**). Además: `docker compose down` **conserva** los datos; `down -v` **los borra**.

9.9 `host not found in upstream "api-b"` en los logs de `gs08-web`

nginx no arranca si `api-b` no existe. **Levantar el stack completo** (`docker compose up -d`), no sólo `web`. Está en el runbook con otros cuatro síntomas frecuentes: `docs/devops/COMANDOS.md` §7.

9.10 El disco `C:` al 94 %

Docker Desktop guarda imágenes y volúmenes en el disco del sistema; el stack completo ocupa ~2,5 GB.

Solución: mover el disco de datos a `D:` desde *Settings* → *Resources* (declarado por @devops).

9.11 Con minikube y driver docker, el cluster no ve las imágenes locales

El daemon del cluster es otro. **Solución:** `minikube image load gs08-api:local gs08-web:local`, o construirlas dentro de `eval $(minikube docker-env)`. (Aplica cuando A-02 se apruebe; hoy no hay cluster.)

10. Verificación rápida (5 comandos)

```
cd /d/dev/equipo/gs08-matriculas

python scripts/validar_infra.py        # 21 OK, exit 0   (sin Docker; es lo que corre el CI en
PR)
python scripts/verificar_stack.py      #  9 OK, exit 0   (stack arriba; requiere los
contenedores)
bash scripts/smoke_api.sh --e2e        # 39 OK, exit 0   (recorrido completo por el proxy; @qa)
bash db/verificacion/ejecutar_verificacion.sh   # modelo + notas + hash del admin, exit 0
kubectl kustomize k8s/                 # 11 objetos      (validación offline de los
manifiestos)
kubectl get pods -n gs08               # 3 pods Running + el Job de migración Complete
```

Corrida de referencia de @documentador, **22/09/2026** (sobre el stack real del compose y el cluster vivo): `curl` por el proxy → API real con `entorno: local`; `/health` y `/metrics` por el proxy → **404**; `/api/v1/health` → **200**; las **4** cabeceras presentes en `/api/`; `docker compose ps` → 6 contenedores `healthy`; `kubectl get pods -n gs08` → 3/3 `Running` y `gs08-migraciones` `Complete 1/1`.

El comando que dice si el contrato está cumplido, criterio por criterio, es el smoke completo (crea y borra sus propios datos):

```
bash scripts/smoke_api.sh --e2e --sin-escritura    # solo lectura
bash scripts/smoke_api.sh --e2e                    # recorrido completo · 39 OK, 0 fallas
```

11. Archivos citados en este manual

```
docker-compose.yml · .env.example · .gitignore
backend/Dockerfile · frontend/Dockerfile · frontend/nginx/default.conf.template
db/init/{01-schema.sql,02-view.sql,03-seed.sql,README.md} · db/verificacion/*
```

```
infra/prometheus/prometheus.yml · infra/grafana/**
k8s/{00..40}*.yaml · k8s/kustomization.yaml
.github/workflows/{ci.yml,cd.yml}
scripts/{validar_infra.py,verificar_stack.py,smoke_api.sh}
docs/analisis/modelo-datos.md · docs/analisis/datos-seed.md · docs/analisis/evidencia/*
docs/devops/plan-contenedores-ci.md · docs/devops/evidencia-sprint0.md ·
docs/devops/COMANDOS.md
docs/alcance-mvp.md · docs/backlog-sprints.md · docs/decisiones.md
docs/qa/plan-pruebas.md · docs/qa/reporte-bugs.md · docs/qa/evidencia/*
README.md · docs/arquitectura.md · docs/actas/*
```

Este manual crece con S1-S3: el contrato de §2.2 pasa de «comprometido» a «verificado» con su ejemplo real de request/response cuando T1.1-T1.4 estén; ahí se agrega la corrida del smoke por criterio y se actualiza §1.

Arquitectura

Entregable: TD-02 de `docs/backlog-sprints.md` · **Redacta:** @documentador · **Fecha:** 21/09/2026

Rama: `main` @ `e051d43` · **Repo:** `D:\dev\equipo\gs08-matriculas`

Este documento **no inventa nada**: cada diagrama sale de un archivo del repositorio o de una salida real, y cada uno dice de dónde. Los datos técnicos que lo respaldan están en `docs/devops/evidencia-sprint0.md` (infraestructura, §1-2 y §5), `docs/devops/plan-contenedores-ci.md` (diseño) y `docs/analisis/modelo-datos.md` §1 (modelo de datos).

Aviso de estado, para leer los diagramas: lo que **existe y corre** hoy es la infraestructura (compose, PostgreSQL con el esquema del legacy, nginx balanceando 2 instancias, Prometheus/Grafana) y el **API real** (19 rutas / 34 operaciones publicadas, probado contra una base limpia: 22 de los 39 criterios con corrida verde). **El SPA es T2.4** y el compose todavía sirve el **stub de humo** en `:8080` (retirarlo es T2.6), así que los diagramas de componentes y de flujo describen la arquitectura ya construida *alrededor* de ese código: el balanceo, el failover y la observabilidad son reales; el SPA del diagrama §1 es el que falta.

Índice

12. Vista de componentes (Docker Compose) (#1-vista-de-componentes-docker-compose)
13. Flujo de una petición: balanceo y failover (#2-flujo-de-una-petición-balanceo-y-failover)
14. Modelo de datos (#3-modelo-de-datos)
15. Despliegue en Kubernetes (#4-despliegue-en-kubernetes)
16. Observabilidad (#5-observabilidad)
17. Decisiones de arquitectura que explican estos diagramas (#6-decisiones-de-arquitectura-que-explican-estos-diagramas)
18. Capa documental, caché, GraphQL y logging centralizado (rúbrica) (#7-capa-documental-caché-graphql-y-logging-centralizado-rúbrica)

1. Vista de componentes (Docker Compose)

Fuente: `docker-compose.yml` (servicios `db`, `mongo`, `redis`, `api`, `api-b`, `web`, perfil `obs` con `prometheus`, `grafana` y `loki`), `frontend/nginx/default.conf.template`, `backend/Dockerfile`, `frontend/Dockerfile`.

Evidencia de que levanta: `docker compose --profile obs ps` → 9 contenedores healthy (`docs/devops/evidencia-sprint0.md` §2; los tres servicios nuevos, verificado el 13/10/2026).

```
flowchart TB
    subgraph HOST["Host Windows 11 + Docker Desktop 29.8.0"]
        direction TB
        subgraph NET["red bridge gs08-net"]
            WEB["gs08-web · nginx 1.31.6-alpine<br/>usuario nginx, escucha 8080<br/>SPA + proxy inverso /api/"]
            API1["gs08-api · FastAPI (Python 3.12)<br/>usuario app uid 10001, :8000"]
            API2["gs08-api-b · misma imagen<br/>segunda instancia, sin puerto al host"]
            DB[("gs08-db · postgres:16.15-alpine3.24<br/>base gs08_matriculas<br/>volumen pgdata")]
            MONGO[("gs08-mongo · mongo:8.0<br/>base gs08_bitacora (bitácora<br/>y boletas) · volumen mongodata")]
            REDIS[("gs08-redis · redis:8-alpine<br/>caché del panel · volumen redisdata")]
            PROM["gs08-prometheus · v3.13.3<br/>perfil obs, :9090"]
            GRAF["gs08-grafana · 12.4.11<br/>perfil obs, :3000<br/>datasources Prometheus + Loki"]
            LOKI["gs08-loki · 3.5.2<br/>perfil obs, :3100<br/>volumen lokidata"]
        end
```

```
    end

    NAV["Navegador / @qa<br/>http://localhost:8080"]
    QA2["Pruebas directas al API<br/>http://localhost:8000 + /docs + /graphql"]

    NAV -->|"GET / (SPA)"| WEB
    NAV -->|"GET /api/v1/... (proxy)"| WEB
    QA2 -->|"health, metrics, Swagger, GraphiQL"| API1
    WEB -->|"upstream gs08_api (round-robin<br/>max_fails=3 fail_timeout=10s)"| API1
    WEB -->|"misma zona de balanceo"| API2
    API1 -->|"SQLAlchemy + psycopg 3"| DB
    API2 -->|"SQLAlchemy + psycopg 3"| DB
    API1 -->|"pymongo: bitácora + boleta emitida"| MONGO
    API2 -->|"pymongo (misma base documental)"| MONGO
    API1 -->|"redis: caché del panel (TTL 60 s)"| REDIS
    API2 -->|"redis: la misma caché"| REDIS
    API1 -->|"logs JSON (push HTTP)"| LOKI
    API2 -->|"logs JSON (push HTTP)"| LOKI
    PROM -->|"scrape /metrics cada 15 s"| API1
    PROM -->|"scrape /metrics cada 15 s"| API2
    GRAF -->|"datasource Prometheus (provisionado)"| PROM
    GRAF -->|"datasource Loki (provisionado)"| LOKI

    subgraph VOL["volúmenes con nombre"]
        V1["pgdata"]
        V2["prometheus_data · retención 15 d"]
        V3["grafana_data"]
        V4["mongodata · redisdata · lokidata"]
    end
    DB -.-> V1
    PROM -.-> V2
    GRAF -.-> V3
    MONGO -.-> V4

    subgraph INIT["db/init/*.sql (solo al crear el volumen)"]
        I1["01-schema.sql · 02-view.sql · 03-seed.sql"]
    end
    I1 -.->|"docker-entrypoint-initdb.d"| DB
```

Puertos publicados al host: `8080` (web), `8000` (api), `5432` (db), `27017` (mongo), `6379` (redis), `9090` (prometheus), `3000` (grafana), `3100` (loki).

`api-b` no publica puerto a propósito: solo lo alcanza nginx por la red interna.

Un contenedor por servicio, imágenes alpine y usuario no root: el porqué está en `docs/seguridad-etica-sostenibilidad.md` §3.

2. Flujo de una petición: balanceo y failover

Fuente: `frontend/nginx/default.conf.template` (bloque `upstream gs08_api` con `zone`, `keepalive` y `proxy_next_upstream`) y `docker-compose.yml` (`API_UPSTREAM` con dos `server` y `max_fails=3 fail_timeout=10s`).

Evidencia real: `docs/devops/evidencia-sprint0.md` §3 (12 peticiones alternando `172.18.0.5`/`172.18.0.6`) y la verificación independiente de @qa (`docs/qa/plan-pruebas.md` §8: 6 peticiones, 3 y 3; y **failover con las 6 en HTTP 200** con `gs08-api-b` apagado).

```
  sequenceDiagram
      autonumber
      participant N as Navegador / QA
      participant W as nginx (gs08-web:8080)
      participant A as gs08-api (:8000)
```

```
    participant B as gs08-api-b (:8000)
    participant D as PostgreSQL (gs08-db:5432)

    N->>W: GET /api/v1/matriculas
    Note over W: upstream gs08_api con zone compartida<br/>-> round-robin entre api y api-b
    W->>A: proxy_pass http://gs08_api
    A->>D: SELECT ... (v_matriculas_detalle)
    D-->>A: filas
    A-->>W: 200 JSON
    W-->>N: 200 JSON + X-Upstream-Addr: 172.18.0.5:8000

    N->>W: GET /api/v1/matriculas (2ª)
    W->>B: proxy_pass (siguiente en el turno)
    B->>D: SELECT ...
    D-->>B: filas
    B-->>W: 200 JSON
    W-->>N: 200 JSON + X-Upstream-Addr: 172.18.0.6:8000

    Note over B: se apaga gs08-api-b (docker stop)
    N->>W: GET /api/v1/matriculas
    W->>B: intento (error de conexión)
    Note over W: proxy_next_upstream error timeout<br/>proxy_next_upstream_tries 2
    W->>A: reintento en la instancia viva
    A->>D: SELECT ...
    A-->>W: 200 JSON
    W-->>N: 200 JSON + X-Upstream-Addr: 172.18.0.6:8000, 172.18.0.5:8000
```

Cómo se reproduce (un comando, sin tocar nada):

```
for i in $(seq 1 6); do curl -s -o /dev/null -D - http://localhost:8080/api/v1/health | grep -i
x-upstream-addr; done
```

Dos detalles que costaron hallazgos y ya están corregidos (`docs/devops/evidencia-sprint0.md` §8):

- Sin `zone gs08_api 64k`; cada **worker** de nginx lleva su propio contador y el round-robin salía desparejo

(todo caía en una instancia). Con la zona, el reparto es 1 a 1.

- Sin `max_fails`/`fail_timeout` + `proxy_next_upstream error timeout`, una instancia que aún no aceptaba

conexiones al arrancar quedaba penalizada y no había reintento. **Nota deliberada:** solo se reintenta en `error timeout`, **no** en 5xx: reintentar un 5xx repetiría escrituras.

3. Modelo de datos

Fuente: `docs/analisis/modelo-datos.md` §1 (traducción de `legacy/script.sql.txt`, MySQL 8 → PostgreSQL 16) y `db/init/01-schema.sql`, `02-view.sql`, `03-seed.sql`.

Evidencia: `bash db/verificacion/ejecutar_verificacion.sh` → **49 comprobaciones OK, exit 0** (incluye 15 pruebas negativas que exigen que la base **rechace** cada regla violada).

```
erDiagram
    usuarios {
        integer id PK "identity BY DEFAULT"
        varchar nombre_usuario UK "3-50, [A-Za-z0-9_.]"
        varchar email UK "formato válido"
        varchar password_hash "bcrypt coste 10, $2y$ del seed"
        varchar nombre_completo "no vacío"
        varchar rol "admin | asistente, default admin"
        boolean estado "default true"
```

```
    timestamptz creado_en "default now()"
}

estudiantes {
    integer id PK "identity"
    varchar codigo UK "E20260001, hasta 20"
    char dni UK "exactamente 8 dígitos"
    varchar nombres "no vacío, indexado"
    varchar apellidos "no vacío, indexado"
    varchar email "nullable, formato válido"
    varchar telefono "nullable"
    date fecha_nacimiento "nullable, mayor a 1900-01-01"
    varchar direccion "nullable"
    boolean estado "false = baja lógica"
    timestamptz creado_en "default now()"
}

cursos {
    integer id PK "identity"
    varchar codigo UK "C101..."
    varchar nombre "no vacío"
    text descripcion "nullable"
    smallint creditos "CHECK 1..10, default 3"
    smallint horas "CHECK 1..1000, default 48"
    boolean estado "default true"
    timestamptz creado_en "default now()"
}

matriculas {
    integer id PK "identity"
    integer estudiante_id FK "ON DELETE CASCADE"
    integer curso_id FK "ON DELETE CASCADE"
    varchar periodo "CHECK AAAA-MM, ej 2026-02"
    date fecha_matricula "no nula"
    varchar estado "activa | retirado, default activa"
    timestamptz creado_en "default now()"
}

notas {
    integer id PK "PROPUESTA, Sprint 3 - T1.5/T3.1"
    integer matricula_id FK "ON DELETE CASCADE"
    varchar tipo "practica | parcial | final"
    smallint numero "mayor o igual a 1"
    numeric nota "0..20"
    date fecha_registro "default CURRENT_DATE"
    varchar observacion "nullable"
}

estudiantes ||--o{ matriculas : "se matricula en"
cursos ||--o{ matriculas : "recibe"
matriculas ||--o{ notas : "se califica con"
usuarios ||..o{ matriculas : "sin FK - sin auditoria (D-08)"
```

Objetos y reglas, en números (`docs/analisis/modelo-datos.md` §1 y §3):

Objeto	Qué tiene	Dónde vive
`usuarios`, `estudiantes`, `cursos`, `matriculas`	4 tablas · **16 CHECK** · **6 UNIQUE** · **2 FK** · **7 índices**	`db/init/01-schema.sql`
`v_matriculas_detalle`	JOIN `matriculas` + `estudiantes` + `cursos`, **24 filas** con el seed	`db/init/02-view.sql`

Seed	1 admin · 12 estudiantes · 7 cursos · **24 matrículas (23 activas + 1 retirada)**	`db/init/03-seed.sql`
La regla central	**`UNIQUE (estudiante_id, curso_id, periodo)`** — un estudiante no repite curso en el mismo periodo	`uq_matriculas_estudiante_curso_periodo`
notas	**propuesta escrita, no aplicada.** Entra en el Sprint 3 (D-01) como esquema **aditivo**: no se toca ninguna de las 4 tablas	`docs/analisis/modelo-datos.md` §7

Dos cosas que el diagrama no dice y conviene tener presentes:

- **`periodo` es `varchar(7)`** con CHECK `^[0-9]{4}-(0[1-9]|1[0-2])$`: en MySQL la columna aceptaba 10

caracteres y el formato lo validaba PHP, así que `2026-13` entraba. Ahora lo rechaza la base.

- **`estado` pasa de `TINYINT(1)` a `boolean`**: en el JSON de la API eso es `true`/`false`, no `1`/`0`.

Avisado a @dev y @qa por @analista (`modelo-datos.md` §8).

4. Despliegue en Kubernetes

Fuente: `k8s/kustomization.yaml` + los 7 manifiestos que referencia; render verificado con `kubectl kustomize k8s/` → **11 objetos** (`docs/devops/evidencia-sprint0.md` §5, reconfirmado el 21/09/2026).

> ***Estado real: manifiestos listos, cluster NO desplegado en esta máquina.*** *`kubectl config current-context` → `error: current-context is not set`; `minikube`/`kind`/`k3s`* ***no están instalados*** *y su instalación espera la decisión del dueño (**A-02**). El criterio `C-37` («3/3 pods Running y respuesta por el ingress») está **bloqueado**, y se escribe así: pendiente — bloqueado por A-02 / T1.6. Lo que sí está validado: el render de kustomize (estructura y referencias cruzadas entre objetos), no el despliegue.*

```
flowchart TB
    subgraph NODE["Nodo local (minikube con driver docker — pendiente de instalar)"]
        subgraph NS["namespace gs08"]
            CFG["ConfigMap gs08-config<br/>DB_HOST, puertos, ENVIRONMENT=k8s, API_UPSTREAM"]
            SEC["Secret gs08-secrets<br/>POSTGRES_PASSWORD, SECRET_KEY,
DATABASE_URL<br/>(plantilla 02-secret.example.yaml, el real va en .gitignore)"]
            PVC["PersistentVolumeClaim gs08-pgdata<br/>2Gi, ReadWriteOnce"]

            subgraph DBSTACK["Capa de datos"]
                DB["Deployment gs08-db (1 réplica, strategy Recreate)<br/>postgres:16.15-
alpine3.24"]
                DBS["Service gs08-db :5432"]
            end

            JOB["Job gs08-migraciones<br/>alembic upgrade head (1 vez por despliegue)"]

            subgraph APPSTACK["Capa de aplicación"]
                APID["Deployment gs08-api (2 réplicas)<br/>RollingUpdate maxUnavailable
0<br/>probes /health, runAsNonRoot 10001"]
                WBD["Deployment gs08-web (2 réplicas)<br/>nginx, probes /healthz"]
                APIS["Service gs08-api :8000 (ClusterIP)"]
                WBS["Service gs08-web :80 (ClusterIP)"]
```

```
            end

            ING["Ingress gs08 (ingressClassName nginx)<br/>gs08.local/ -> gs08-
web:80<br/>gs08.local/api -> gs08-api:8000"]
        end
    end

    USR["Navegador<br/>http://gs08.local"]
    USR --> ING
    ING --> WBS
    ING --> APIS
    WBS --> WBD
    APIS --> APID
    APID --> APIS
    WBD -->|"API_UPSTREAM = server gs08-api:8000"| APIS
    APID --> DBS
    JOB --> DBS
    DB --> PVC
    CFG -.-> APID
    CFG -.-> WBD
    CFG -.-> DB
    SEC -.-> APID
    SEC -.-> DB
```

Los 11 objetos (salida de `kubectl kustomize k8s/`, en orden de render):

#	Kind	Nombre	Para qué
1	Namespace	`gs08`	todo vive en su namespace
2	ConfigMap	`gs08-config`	configuración no sensible
3	Service	`gs08-api`	ClusterIP `:8000`
4	Service	`gs08-db`	ClusterIP `:5432`
5	Service	`gs08-web`	ClusterIP `:80` → `8080` del contenedor
6	PersistentVolumeClaim	`gs08-pgdata`	2Gi, `ReadWriteOnce`
7	Deployment	`gs08-api`	2 réplicas, `RollingUpdate` sin caídas
8	Deployment	`gs08-db`	1 réplica, `Recreate` (RWO no se monta dos veces)
9	Deployment	`gs08-web`	2 réplicas
10	Job	`gs08-migraciones`	`alembic upgrade head` una vez por despliegue
11	Ingress	`gs08`	un solo host para SPA y API

Nota de coherencia (D-14): en el cluster el DDL inicial lo aplica **Alembic**, no `db/init/` — el Job `gs08-migraciones` corre `alembic upgrade head`. Por eso la revisión inicial de Alembic se genera **desde** `db/init/` y hay una sola fuente de verdad del esquema.

Comandos exactos para levantarlo cuando se apruebe minikube: `docs/devops/COMANDOS.md` §5.

5. Observabilidad

Fuente: `infra/prometheus/prometheus.yml`, `infra/grafana/provisioning/*`, `infra/grafana/dashboards/gs08-matriculas.json`, `docs/devops/plan-contenedores-ci.md` §5.

Evidencia: `docs/devops/evidencia-sprint0.md` §4 (3 targets `up`, consultas de los paneles con tráfico real: 0,517 req/s · 3,12 % 5xx · 8,13 % 4xx · p95 0,095 s) y la corrida de `verificar_stack.py` del 21/09/2026 22:44: *«targets Prometheus: 3 targets, caidos: ninguno»*, *«paneles Grafana con datos: 9 paneles revisados, sin datos: ninguno»*.

```
flowchart LR
    subgraph APIS["Instancias del API"]
        A["gs08-api :8000"]
        B["gs08-api-b :8000"]
    end

    subgraph OBS["Perfil obs (docker compose --profile obs up -d)"]
        P["Prometheus v3.13.3 :9090<br/>scrape_interval 15s<br/>retención 15d"]
        G["Grafana 12.4.11 :3000<br/>usuario admin"]
    end

    subgraph DASH["Dashboard API RED (método Rate-Errors-Duration)"]
        D1["1 Disponibilidad (up)"]
        D2["2 Peticiones/segundo"]
        D3["3 Errores 5xx (%)"]
        D4["4 Errores 4xx (%)"]
        D5["5 Tráfico por instancia"]
        D6["6 Latencia p50/p95/p99"]
        D7["7 Respuestas por estado"]
        D8["8 Top 5 endpoints"]
        D9["9 p95 por endpoint"]
    end

    A -->|"/metrics"| P
    B -->|"/metrics"| P
    P -->|"datasource (provisionado)"| G
    G --> D1
    G --> D2
    G --> D3
    G --> D4
    G --> D5
    G --> D6
    G --> D7
    G --> D8
    G --> D9
    GRAFJSON["infra/grafana/dashboards/gs08-matriculas.json<br/>ES la fuente: se editó el título y Grafana lo<br/>reflejó en 30 s (v2 -> v3)"] -.->|"provisioning automático"| G
```

Lecturas clave de los paneles (`docs/devops/plan-contenedores-ci.md` §5):

- **Panel 1 (up):** 2/2 vivas = todo OK; una **CAÍDO** con tráfico en la otra demuestra que nginx y el servicio sobreviven — es la vista en vivo del failover del §2.
- **Panel 3 vs 4:** 5xx debe ser **0**; un 4xx alto con 5xx en 0 es buena señal: significa que las reglas de negocio están rechazando lo que deben (401, 404, 409).
- **Panel 6:** criterio del MVP, p95 < 0,5 s (medido: 0,095 s con el stub; se repite con el API real, T2.6).

Limitación declarada: no hay paneles de PostgreSQL ni de nginx. Ninguno de los dos expone métricas en formato Prometheus sin un exporter propio; el job de nginx que existía se eliminó porque fallaba (`ok\n` no es formato Prometheus). Agregar `postgres-exporter` y `nginx-prometheus-exporter` es Sprint 1 si el tiempo alcanza (`docs/devops/plan-contenedores-ci.md` §5 y §7).

6. Decisiones de arquitectura que explican estos diagramas

Citadas **tal cual** de `docs/decisiones.md` (no se reescriben, D-20):

Decisión	Qué fija	Dónde se ve en este documento
D-00	Stack: FastAPI + PostgreSQL 16 + Vue 3 (Vite) + Tailwind, Docker, GitHub Actions, Kubernetes local, Prometheus + Grafana. Se reimplementa el legacy, no se migra	§1, §4, §5
D-14	Una sola fuente de verdad del esquema: la revisión inicial de Alembic se genera desde `db/init/`	§3 y §4 (Job de migraciones)
D-17	Health por el proxy en `/api/v1/health`; `/metrics` **directo** en `:8000`; nginx **404** en `/health` y `/metrics`	§1, §2, §5
D-04	Kubernetes local con minikube y driver `docker`; instalarlo requiere aprobación del dueño	§4
D-11	Buscador con `ILIKE` (+ `unaccent` para tildes), nunca `LIKE`	§3 (nota de `periodo`/buscador)
D-02	Borrar estudiante o curso con matrículas no borra en cascada silenciosa: `409` con el conteo, y `204` solo con `?confirmar=true`	§3 (FK `ON DELETE CASCADE` + regla de la API)
D-01	Las notas **sí** entran al MVP, en el Sprint 3, como esquema aditivo	§3 (bloque `notas`, marcado como propuesta)
D-08	Sin auditoría (`creado_por`, triggers) en la v1	§3 (relación punteada `usuarios` ↔ `matriculas`)

7. Capa documental, caché, GraphQL y logging centralizado (rúbrica)

Fuente: `backend/app/mongo.py`, `backend/app/cache.py`, `backend/app/graphql.py`, `backend/app/observabilidad.py`, `backend/app/boleta.py`, `backend/app/routers/auditoria.py`, `infra/loki/loki-config.yml`, `infra/grafana/provisioning/datasources/loki.yml`. Decisión: **D-28**.

La consigna del trabajo colaborativo pide «bases de datos híbridas (PostgreSQL + MongoDB)», «APIs REST y GraphQL», «OAuth2, JWT y roles RBAC» y «caching distribuido (Redis) y logging centralizado». Se resolvió

añadiendo capas al API en vez de reescribirlo: el SPA, el balanceo de nginx y el despliegue no se mueven.

```
flowchart LR
    H["Navegador (SPA React)"]
```

```
    Q["GraphiQL / cliente GraphQL"]
    subgraph API["gs08-api y gs08-api-b (sin cambios de despliegue)"]
        MW["Middleware: X-Request-ID,<br/>log JSON, bitácora e invalidación"]
        REST["Routers REST /api/v1"]
        GQL["Strawberry /graphql"]
        BOL["app/boleta.py: una sola fórmula"]
    end
    PG[("PostgreSQL 16<br/>datos relacionales")]
    MG[("MongoDB 8<br/>bitacora · boletas_snapshot")]
    RD[("Redis 8<br/>gs08:panel (TTL 60 s)")]
    LK[("Loki 3.5")]
    GF["Grafana (Explore)"]
    H -->|"REST con JWT"| MW
    Q -->|"POST /graphql con JWT"| MW
    MW --> REST
    MW --> GQL
    REST --> BOL
    GQL --> BOL
    REST --> PG
    GQL --> PG
    MW -->|"cada escritura"| MG
    REST -->|"boleta emitida"| MG
    MW -->|"invalida tras escribir"| RD
    REST -->|"lee y escribe la caché"| RD
    MW -->|"log JSON"| LK
    LK --> GF
```

Reglas que hacen que esto no sea decorativo:

19. **Una sola fórmula de boleta** (`app/boleta.py`): REST y GraphQL calculan igual, así que no pueden dar números

distintos entre sí. Mismo criterio que D-14 con el esquema.

20. **La bitácora la escribe el middleware**, no cada endpoint: ninguna operación futura queda sin auditar por

olvido, y las credenciales se enmascaran (`***`) antes de llegar a Mongo.

21. **Degradación limpia:** sin `MONGO_URL` o `REDIS_URL` el API responde igual (sin bitácora / sin caché). Es lo

que permite que el CI —que solo levanta PostgreSQL— siga corriendo las 103 pruebas.

22. **La caché se invalida sola** en cada escritura correcta: no queda un panel viejo después de matricular.
23. **Logs centralizados sin Promtail:** el API empuja el JSON a Loki por su API HTTP, así funciona igual en

Windows (Docker Desktop) y en el VPS, sin montar el socket de Docker.

Medido el 14/10/2026 (stack local, `docs/decisiones.md` D-28, D-29 y D-31): **121 pruebas y 90 % de cobertura**; caché del panel **54,2 ms → 13,5 ms (4,0x)** con 20 000 estudiantes y 20 000 matrículas; bitácora con las escrituras reales del recorrido y **sin la contraseña**; Loki con la serie `{servicio="gs08-api"}`; y la autenticación OAuth2 con **16 scopes** (token restringido → 403 en lo que no le toca).

7.1 Flujo de autenticación: OAuth2 password grant con scopes (D-29)

```
  sequenceDiagram
      participant S as SPA / cliente
      participant A as API (gs08-api)
```

```
participant P as PostgreSQL
S->>A: POST /api/v1/auth/token (grant_type=password, username, password, scope?)
A->>P: busca el usuario y verifica el hash bcrypt
alt credenciales malas o cuenta desactivada
    A-->>S: 400 {"error":"invalid_grant"}
else correctas
    A->>A: rol -> scopes (app/scopes.py)
    A-->>S: 200 {access_token, token_type, expires_in, scope}
end
S->>A: GET /api/v1/matriculas (Authorization: Bearer …)
A->>A: requiere_scope("matriculas:leer") revisa el `scp` del token
alt al token le falta el scope
    A-->>S: 403 {"error":"insufficient_scope"} + WWW-Authenticate
else lo tiene
    A->>P: consulta
    A-->>S: 200 […]
end
```

El servidor de autorización es el propio API: **no hay contenedor, RAM ni dominio extra** (el SPA es cliente propio). El token lleva los scopes en `scp` y cada endpoint pide el suyo; el reparto por rol está en `docs/manual-tecnico.md` §2.6.

Archivos de los que sale este documento: `docker-compose.yml`, `frontend/nginx/default.conf.template`, `frontend/Dockerfile`, `backend/Dockerfile`, `db/init/*.sql`, `k8s/*.yaml`, `infra/prometheus/prometheus.yml`, `infra/grafana/**`, `docs/devops/evidencia-sprint0.md`, `docs/devops/plan-contenedores-ci.md`, `docs/analisis/modelo-datos.md`, `docs/decisiones.md`, `docs/qa/plan-pruebas.md`.

Verificado por @documentador el 21/09/2026 22:44: `kubectl kustomize k8s/` (11 objetos), `docker ps` (7 contenedores), `python scripts/validar_infra.py` (21 OK), `python scripts/verificar_stack.py` (9 OK, exit 0).

Registro de decisiones

Mantiene: @pm · **Última actualización:** 21/09/2026 · **Sprint 0** · Movida desde `docs/` a `docs/` para que viva junto al resto de entregables (ver D-20).

Cada decisión queda con fecha, motivo y estado. `✅ **Cerrada**` = se ejecuta tal cual está escrito.

`☐ **Abierta**` = falta un dato o la respuesta del dueño. Si algo de acá contradice a un documento anterior, manda esta tabla.

#	Fecha	Decisión	Por qué	Estado
D-00	21/09/2026	**Stack: FastAPI + PostgreSQL 16 + Vue 3 (Vite) + Tailwind, contenedores Docker, GitHub Actions, Kubernetes local y Prometheus + Grafana.** Se reimplementa el legacy PHP+MySQL, no se migra	Es el stack exigido por el curso GS08 y el que permite demostrar lo que el legacy no tiene: contrato de API probado, esquema con reglas en la base, CI, despliegue reproducible y métricas. Se conserva del legacy **el modelo de datos y la UX**, no el código	✅ Cerrada
D-01	21/09/2026	**Las notas SÍ entran al MVP**, en el Sprint 3, como esquema **aditivo**: `db/init/04-notas.sql` + `v_notas_detalle`. **No se toca ninguna de las 4 tablas existentes**	El proyecto del curso se llama «Matrículas **y Notas**» y el legacy no tiene notas: dejarlas fuera abre un hueco en la sustentación. El DDL ya está escrito y verificado en `modelo-datos.md` §7, es aditivo, y entra al final para no moverle el piso a @dev ni a @qa a mitad de sprint	✅ Cerrada
D-02	21/09/2026	**Borrar estudiante o curso con matrículas NO borra en cascada silenciosa.** `DELETE` sin `?confirmar=true` responde **409** con `{"detail":"...","matriculas":N}`; con `?confirmar=true` borra en cascada (204)	La cascada del legacy funciona pero es invisible: el usuario se entera después. El conteo previo convierte una pérdida de datos en una decisión. La baja lógica (`estado=false`) sigue siendo el camino normal	✅ Cerrada
D-03	21/09/2026	**Nota del curso = media aritmética** de las notas registradas (2 decimales). **Promedio del periodo = media ponderada por créditos** de los cursos con notas. Curso sin notas → `null`, no cuenta como 0	El legacy no tiene ninguna política de notas, así que no hay pesos que heredar. Inventar «práctica 30 % / parcial 30 % / final 40 %» sería un dato falso en el informe. La media aritmética es la regla más simple, explicable y verificable; ponderar por créditos sí es práctica	✅ Cerrada

			estándar y el dato (`cursos.creditos`) existe	
D-04	21/09/2026	**Kubernetes local con minikube, driver `docker`** (addon ingress). **No** k3s, **no** kind salvo plan B. **Instalarlo requiere aprobación del dueño**: es software nuevo en la máquina	Docker Desktop 29.8.0 ya corre con 12 GB y el driver docker de minikube no necesita Hyper-V ni WSL. Trae addon de ingress y dashboard: sirve para la demo de la sustentación. k3s en Windows exige WSL2 + systemd y es el camino largo. Con driver docker, las imágenes locales se cargan con `minikube image load` (ya anotado como problema conocido en TD-04). Verificado por @pm y @devops: minikube **no está instalado** hoy	✅ Cerrada (decisión) · ☐ pendiente la aprobación de instalación (A-02)
D-05	21/09/2026	**La rama es `main`** (ya está renombrada en el repo local) y los workflows disparan en `push`/`pull_request` a `main` + `workflow_dispatch`. Se **ajusta el workflow, no la rama**	GitHub usa `main` por defecto y el push al repo público es el último paso: renombrar la rama local es gratis hoy y evita que el primer push no dispare nada. Verificado dos veces (`git branch --show-current` → `main`, y `ci.yml`/`cd.yml` apuntan a `main`)	✅ Cerrada
D-06	21/09/2026	**No se registran notas de una matrícula en estado `retirado`** → **409** (RN-14). Retirar una matrícula **no borra** las notas ya registradas	Coherente con la baja lógica: retirar saca del KPI pero conserva el historial. Calificar a alguien que se retiró es un error de digitación, no una función	✅ Cerrada
D-07	21/09/2026	**Se pagina (25 por página)**, aunque el legacy muestra todo sin `LIMIT/OFFSET`	Es una mejora declarada, no una regresión: con 100.000 filas el buscador sin índice tarda 112 ms (medido por @analista) y un listado sin paginar es peor. `page_size` fuera de rango → **422**	✅ Cerrada
D-08	21/09/2026	**Sin auditoría** (`creado_por`, `actualizado_por`, triggers) en la v1	El legacy no la tenía y el curso no la pide. Agregarla significa tabla, trigger y pruebas: costo real sin requisito detrás. Queda anotada en	✅ Cerrada

			`modelo-datos.md` §9 como supuesto	
D-09	21/09/2026	**Usuarios: baja lógica** (`estado=false`) como operación normal; el borrado duro se mantiene solo con `?confirmar=true` y nunca sobre la propia cuenta (**409**)	El legacy borraba con `DELETE` y no se podía eliminar a sí mismo. Mantengo la protección y cambio el camino normal a desactivar: un usuario con matrículas o notas registradas no debe desaparecer	✅ Cerrada
D-10	21/09/2026	**Sin portal de estudiante ni rol docente** en la v1	No existe tabla de usuarios-estudiante ni de docentes en el legacy; agregar ambos roles duplica las pantallas y no aporta al problema de gestión de matrículas y notas que se está resolviendo	✅ Cerrada
D-11	21/09/2026	**El buscador usa `ILIKE`** (y `unaccent` si se quieren tildes), nunca `LIKE`	Medido: `LIKE '%quispe%'` → 0 filas e `ILIKE` → 2 sobre los mismos datos. Copiar el `LIKE` del legacy deja el buscador mudo sin dar error. Índice trigram: es un «si algún día crece» (112 ms → 0,09 ms con 100.000 filas), no del MVP	✅ Cerrada
D-12	21/09/2026	**El hash bcrypt `$2y$` del seed se deja tal cual**; la API lo verifica con `bcrypt`, y **nadie** lo «arregla» por el quirk de pgcrypto	Verificado: `checkpw(b'Admin123!', hash)` → `True`; pgcrypto solo coincide normalizando el prefijo a `$2a$`. Regenerarlo perdería la trazabilidad con el legacy	✅ Cerrada
D-13	21/09/2026	**Los ids con huecos son correctos**: no se tocan las secuencias y el seed cierra con `setval`	`nextval` no se revierte con ROLLBACK y un INSERT rechazado consume el id (verificado: tras 3 fallos el siguiente curso válido fue 11). No es bug	✅ Cerrada
D-14	21/09/2026	**Una sola fuente de verdad del esquema**: la revisión inicial de Alembic se genera **desde `db/init/`**, y el Job `gs08-migraciones` aplica esa migración	Hoy hay dos DDL del mismo esquema (scripts psql y Job de Alembic); dos verdades es un bug esperando su turno. Se unifica en la migración de Alembic generada desde	✅ Cerrada y **verificada el 21/09/2026**: `db/init/` y la revisión `0001_esquema_inicial` dejan el **mismo esquema — 170/170 líneas de catálogo idénticas, exit 0**

			el esquema ya verificado	(`docs/analisis/verificacion-d14-alembic.md` y `docs/dev/evidencia/d14-comparar-con-alembic.txt`), y `alembic downgrade base` deja la base limpia
D-15	21/09/2026	**Entregables finales en `OneDrive\RAR\ISAM\<periodo>`** (informe, PPT, manuales) y el código en `D:\dev\equipo\gs08-matriculas` + repo público	Es donde se entregan los trabajos de ISAM y donde el dueño los encuentra	✅ Cerrada
D-16	21/09/2026	**Sin `Pinia` ni librería de componentes**: composables + Tailwind	Lo que pide el curso es Vue 3 + Vite + Tailwind; meter una librería de componentes agrega peso y estilos ajenos al diseño	✅ Cerrada
D-17	21/09/2026	**Salud y métricas:** la salud del API se consulta por el proxy en **`/api/v1/health`**; **`/metrics` se lee directo del API en `http://localhost:8000/metrics`** (Prometheus no scrapea a través del balanceador: mezclaría las métricas de las dos instancias); y nginx responde **404** —no el `index.html`— en `/health` y `/metrics`. `/healthz` sigue siendo el healthcheck del contenedor nginx	Cierra BUG-05 de @qa: hoy `/metrics` por el proxy devuelve **200 con el `index.html` del SPA**, así que un chequeo de monitoreo que mire solo el código HTTP da por bueno un `/metrics` que no trae métricas. Y scrapear a través del balanceador rompe la atribución por instancia	✅ Cerrada (la ejecuta @devops en TC-03)
D-18	21/09/2026	**El alcance tiene 39 criterios de aceptación, numerados `C-01`…`C-39` en `alcance-mvp.md` §4, y ese es el único denominador** de «100 % de cobertura» (T4.2). Yo dije 35 en la sala: **el número real es 39**	Cierra BUG-04 de @qa: con dos denominadores la cobertura no se puede verificar. Mi conteo estaba mal porque mi grep no veía los criterios escritos en línea (ej. `Listado paginado:` ✅ …). Con ID único, cada caso de prueba puede citar el criterio que cubre	✅ Cerrada
D-19	21/09/2026	**`verificar_stack.py`: una serie de Prometheus ausente es ADVERTENCIA, no falla**	Cierra BUG-01 de @qa: en un entorno recién levantado la serie `status=~"5.."` no existe y Prometheus devuelve vacío (no 0), así que el script salía con exit 1 sin ningún 500 real. Un verificador que falla en el camino limpio es peor que no tenerlo: la demo arranca con `down -v` + `up`	✅ Cerrada (la ejecuta @devops en TC-01)

D-20	21/09/2026	**La documentación vive en `docs/` (raíz) y esta tabla manda en `docs/decisiones.md`.** El conjunto documental del proyecto es: `README.md` (raíz), `docs/arquitectura.md`, `docs/manual-usuario.md`, `docs/manual-tecnico.md`, `docs/actas/`, `docs/decisiones.md`, `docs/seguridad-etica-sostenibilidad.md` + los docs técnicos de `docs/devops/`, `docs/analisis/`, `docs/qa/` y `docs/dev/`, y el **paquete académico** de `docs/informe/` (borradores DOCX/PDF/PPTX, D-22) **No se duplica ningún documento**: `docs/decisiones.md` se cita, no se reescribe	Pedido del dueño (21/09/2026) y punto planteado por @documentador: el archivo estaba en `docs/pm/decisiones.md` mientras el resto de entregables va en la raíz de `docs/`. Una sola ruta evita dos versiones de la misma tabla	✅ Cerrada
D-21	21/09/2026	**Ante una discrepancia entre el alcance y el seed, mandan los datos del seed** (y se corrige el alcance): se corrigió el correo a `admin@horizonte.edu.pe` y el DNI de ejemplo a `45123456`	Cierra BUG-02 y BUG-03 de @qa. El seed viene del legacy y está verificado con 49 comprobaciones, y ya lo citan la integración de @devops y el E2E de @qa: cambiarlo obligaría a regenerar tres evidencias por dos datos de ejemplo. Fuente de los valores: `docs/analisis/datos-seed.md`	✅ Cerrada
D-22	21/09/2026	**El DOCX y el PPTX de entrega académica los cierra el dueño**; @documentador entrega el material fuente y el borrador armado con las tablas y salidas reales	Lo pidió el dueño así: la versión académica final (portada, formato, normas de la facultad) es suya. El borrador sale de los documentos con evidencia, no de datos nuevos	✅ Cerrada

D-23	**21/09/2026**	**Entregable académico del GS08 (datos del dueño, 21/09/2026): grupo de 3 integrantes — Arteaga Rullier, Roberto (Scrum Master + DevOps), Díaz Cárdenas, Jorge Luis (Backend), Inocencio Gargate, Dalcir (Frontend + Documentador). Docente: Dr. Jack Harry Peralta Cáceres. Informe y exposición en español, con firma grupal; la exposición son 10 láminas concisas, una idea por lámina; cada integrante documenta y expone su bloque**	**Es el formato que exige la entrega del curso. De aquí salen el reparto por persona (sección «Entregable académico» del backlog) y el formato del DOCX/PPTX de TD-09 (10 láminas exactas). Los nombres van en el informe y en la carátula porque la firma es**	✅ **Cerrada**

			grupal	
D-24	21/09/2026	**El control de D-14 es uno solo: `db/verificacion/comparar_con_alembic.sh`** (compara el catálogo de una base con `db/init/` contra una migrada con Alembic; exit 0 = iguales, 2 = la foto falló). @dev borró sus dos scripts duplicados. Se correrá en el job `infra` del CI	Dos comparadores del mismo esquema es justo el problema que D-14 vino a resolver. El de @analista es el más completo (incluye `md5` de los cuerpos de funciones y las 34 constraints con su definición) y ya dio 170/170. Solo necesita la imagen del backend: no depende del SPA	✅ Cerrada
D-25	21/09/2026	**Las cifras de los documentos académicos salen de la fuente dueña, nunca de memoria**: API → @dev (`/openapi.json`), criterios → @qa (matriz), infraestructura → @devops. Toda corrección de cifra se aplica **en las 4 versiones a la vez** (DOCX, PDF, PPTX, PDF)	El paquete académico ya publicó «28 endpoints» (son **32**: 16 rutas / 30 operaciones + `/` y `/metrics` — lo conté yo con `curl` el 21/09 23:21) y «4 criterios incumplidos» cuando los dos `500` ya estaban cerrados a las 23:09. Mismo patrón que los desbordes del PPTX: cifras copiadas antes del último cambio. Con un dueño por cifra, la corrección es de un lugar y no de cuatro	✅ Cerrada
D-26	21/09/2026	**Toda corrida de pruebas declara el commit (`git rev-parse --short HEAD`) o el `md5` del artefacto que probó, y se re-corre cuando el artefacto cambia.** Los `500` de BUG-08/BUG-09 se reportaron contra un venv con el código de las **23:07** y estaban arreglados desde las **23:09**: el reporte era correcto para el artefacto probado y falso para el repo	Evita las dos caras del mismo error: el **falso rojo** (reportar un bug ya cerrado) y el **falso verde** que encontró @analista (un «IDÉNTICOS» comparando dos fotos de una línea). En los dos casos el verificador no estaba mintiendo: no sabía qué había probado. Con el hash en el mensaje, cualquiera reproduce contra el mismo artefacto	✅ Cerrada
D-27	13/10/2026	**El frontend se migra de Vue 3 a React 19 + Vite + Tailwind.** El resto de D-00 sigue igual (FastAPI + PostgreSQL 16, Docker, GitHub Actions, Kubernetes local, Prometheus + Grafana)	Lo pide la consigna del trabajo colaborativo de Implementación de Soluciones para Plataformas Web, que nombra React con Tailwind. Es la vía de menor costo y no mueve el despliegue: la SPA	✅ Cerrada

			está desacoplada del API (todo pasa por `/api/v1`), así que se reescriben solo las vistas y el build sigue siendo estático servido por nginx, con la misma imagen y el mismo Coolify. En la misma pasada el frontend pasó de 4 vistas a **consumir las 30 operaciones del contrato** (cursos, usuarios, notas y boleta incluidos) y sumó `eslint`, pruebas con `vitest` y convenciones documentadas en `frontend/README.md`. Verificado contra el stack local: lint, 9 pruebas, build, imagen Docker reconstruida, recorrido en el navegador y boleta con 14/16/18 → **16.00**	

D-28	**13/10/2026**	**Se aplica la rúbrica al API: MongoDB (segunda base documental), Redis (caché distribuida), GraphQL y logging centralizado. El núcleo sigue siendo FastAPI + PostgreSQL 16. MongoDB guarda la bitácora de toda escritura y la boleta emitida (colecciones `bitacora` y `boletas_snapshot`); Redis cachea el panel (TTL 60 s) y el mismo middleware que audita invalida la caché; GraphQL se monta en `/graphql` sobre las mismas consultas y la misma fórmula de boleta (`app/boleta.py`, compartida con el REST); los logs son JSON con `pedido_id` y usuario y se empujan a Loki (perfil obs) cuando `LOKI_URL`**	**Es el ítem «Backend seguro y eficiente — APIs + BD híbridas» de la consigna del trabajo colaborativo (PostgreSQL + MongoDB, REST y GraphQL, OAuth2/JWT/RBAC, caching distribuido con Redis y logging centralizado). Se elige añadir capas al API actual en vez de reescribirlo: el 20 % de la rúbrica se cubre sin mover el despliegue (misma imagen, mismo Coolify). La fórmula de la boleta se extrajo a `app/boleta.py` para que REST y GraphQL no puedan dar números distintos (mismo motivo que D-14)**	✅ **Cerrada y verificada el 13/10/2026: 103 pruebas en verde, 89 % de cobertura (`ruff check` y `ruff format --check` limpios); caché medida 54,2 ms → 13,5 ms (4,0x) con 20 000 estudiantes y 20 000 matrículas (`redis-cli ttl gs08:panel` → 60 s y la clave desaparece tras cada escritura); bitácora con los documentos reales del recorrido (login y alta) sin la contraseña (queda `***`); consulta GraphQL del panel y de huaman sin tilde (401 sin token); Loki con la serie `{servicio="gs08-api"}` y la línea del**

		está definida. Las dos piezas nuevas degradan solas: sin `MONGO_URL`/`REDIS_URL` el API responde igual (sin bitácora / sin caché), que es lo que mantiene verde al CI, que solo trae PostgreSQL		**log. Queda abierto A-04 (Express.js)**

D-29	**13/10/2026**	**La autenticación pasa a OAuth2: `POST /api/v1/auth/token` (password grant, RFC 6749 §4.3) con scopes por rol, dentro del propio API. El rol deja de autorizar por sí solo: el rol asigna scopes (`app/scopes.py`) y cada endpoint exige el suyo con `requiere_scope("matriculas:escribir")`. `POST /api/v1/auth/login` se mantiene como alias en JSON para que el SPA no cambie (y su token ya lleva los scopes). El `client_id`/`client_secret` son opcionales (el SPA es cliente público) pero si llegan se validan; los errores salen con la forma del estándar (`invalid_client`, `invalid_grant`, `invalid_scope`, `unsupported_grant_type`) y el 403 por permiso lleva `WWW-Authenticate: Bearer error="insufficient_scope"`. Sin dependencias nuevas: se usa el `Form` y el `OAuth2PasswordBearer` de FastAPI**	**Es el ítem «autenticación y autorización con OAuth2, JWT y roles RBAC» de la consigna del trabajo colaborativo. Se implementa el flujo dentro del API (no se monta Keycloak ni un proveedor externo) porque el SPA es cliente propio: eso cubre el estándar sin sumar contenedores, RAM ni dominios al VPS. El password grant es OAuth 2.0; OAuth 2.1 lo desaconseja para clientes de terceros, no para este caso. Si el docente pidiera el flujo con redirección (`/authorize` + PKCE) queda anotado como la extensión natural, 1-1,5 días más**	✅ **Cerrada y verificada el 13/10/2026: 119 pruebas en verde, 90 % de cobertura, `ruff check` y `ruff format --check` limpios. En el stack local: token de admin con 16 scopes (`expires_in` 3600), token restringido a `estudiantes:leer` → GET 200 / POST 403 con `insufficient_scope`; `volcan:explotar` → 400 invalid_scope; clave mala → 400 invalid_grant; `grant_type` raro → 400 unsupported_grant_type; Swagger muestra el botón Authorize con los 16 scopes y el candado por endpoint (`/openapi.json` v0.3.0, 20 rutas / 35 operaciones); el login del SPA por nginx sigue en 200 y la bitácora guarda la petición de token con la contraseña y el secreto en `***`**
D-30	13/10/2026	**El SPA se trabaja para accesibilidad (WCAG 2.1 AA) y suma la pantalla de Auditoría.** Accesibilidad: foco atrapado en los diálogos (entra al abrir, no se escapa con Tab, vuelve al cerrar), Escape, `aria-labelledby`, enlace **«Saltar al contenido»**, foco al contenido al	El ítem del frontend de la consigna pide «criterios de UX/UI inclusivo (WCAG, ARIA)»	✅ Cerrada y **verificada el 13/10/2026**: **22 pruebas del SPA en verde** (5 archivos, incluida la batería de accesibilidad con `axe-core`), `eslint` limpio, `vite build`

		cambiar de sección con anuncio `aria-live`, `aria-current="page"` en el menú, `<caption>` y `scope="col"` en las tablas, `aria-busy` en las cargas, `aria-label` en los botones de cada fila («Editar a Huamán, María»), foco siempre visible (`:focus-visible`), `prefers-reduced-motion` y `aria-invalid/aria-describedby` en el login. Pantalla de Auditoría (`/auditoria`, solo admin): lee la bitácora de MongoDB con filtros y muestra la boleta archivada de un estudiante y periodo	con evidencia, y el backend pide «documentación » de la API: la pantalla hace visible en la demo la capa documental (bitácora y boleta archivada) sin abrir Mongo por consola. La accesibilidad se mide, no se declara: pruebas con `axe-core` en la suite + contraste calculado sobre el CSS compilado	y `build-storybook` OK; contraste **14/14 pares AA (4.5:1)** medido sobre el CSS compilado (texto del cuerpo 13.37:1, secundario 4.77:1, botón principal 5.85:1) y anillo de foco 5.93:1 (≥3:1); contenedor reconstruido y verificado en el navegador
D-32	14/10/2026	**El despliegue del VPS pasa a ser CD de GitHub Actions.** El entorno real del proyecto es el VPS del grupo con Coolify (compose + Traefik + Let's Encrypt): eso es la «nube» del trabajo, con HTTPS y dominio propios. Hasta ahora el CI corría en Actions (5 jobs) y publicaba las imágenes en GHCR, pero el despliegue se pedía a mano; el job `desplegar_vps` de `.github/workflows/cd.yml` lo dispara al terminar el CI en verde sobre `main`, espera el estado `finished` de Coolify y verifica producción (health 200 y el bundle servido)	La consigna pide «automatización de despliegue con Docker, GitHub Actions y Kubernetes»: Actions ya hacía la integración continua y ahora también la entrega. Se reutiliza el patrón de los otros dos despliegues del repo (Kubernetes y cPanel): apagado por una variable hasta que el dueño configure el secreto, así el pipeline no se rompe mientras falte	✅ Escrito y validado el 14/10/2026. Además se corrigió el disparador del CD: apuntaba al workflow «CI» y el nombre real es «CI · Integración continua», así que **el CD nunca había corrido**; ahora sí corre (job verde y los tres despliegues saltados por sus variables). Gate `VPS_DEPLOY_ENABLED`). Queda **encendible**: el dueño agrega el secret `COOLIFY_TOKEN` y la variable `VPS_DEPLOY_ENABLED=true`. El despliegue manual equivalente ya se hizo esa noche: `finished` en 57 s, SPA React sirviendo, health 200, login con 16 scopes y bitácora activa
D-31	14/10/2026	**Lo que encontró la verificación en el navegador: reintento con enfriamiento y dos verdades del SPA.** (1) *Reintento*: si Mongo o Redis no respondían en el primer uso, la bitácora o la caché quedaban apagadas **para siempre** en ese proceso (visto en vivo: una instancia del API respondía `activa:false`); ahora se reintenta cada `ENFRIAMIENTO_SEGUNDOS=30` s y se recupera sola. (2) *La pantalla de Auditoría no miente*: ante un fallo de la consulta decía «la bitácora está apagada» —otra cosa—; ahora distingue «Consultando…» y «No se pudo	Los tres casos solo aparecen con el stack real, no con los dobles; el estado inicial de la vista (`activa:false`) convertía un error en una afirmación falsa, y eso en una	✅ Verificado el 14/10/2026: **121 pruebas del backend (90 %)** y **25 del SPA** (5 archivos), `ruff` y `eslint` limpios, contenedores `api`, `api-b` y `web` reconstruidos; `GET /api/v1/auditoria` con `activa:true` **en las dos instancias** del API (visto con la cabecera `X-Upstream-Addr` de nginx) y la bitácora con 29

		consultar» de «apagada», y solo afirma lo que el API confirmó. (3) *Sesión vencida*: un 401 en cualquier ruta protegida olvida la sesión y vuelve al acceso (`/auth/login` queda fuera: ahí un 401 es clave equivocada)	demo se nota	documentos

Decisiones abiertas

#	Fecha	Qué falta	De quién depende	Impacto si no llega
A-01	21/09/2026	**Fecha de entrega y fecha de exposición del GS08**	@user	No puedo fechar los 4 sprints ni decir qué se corta si el tiempo no alcanza. El alcance y el reparto no dependen de esto: el trabajo arranca igual
A-02	21/09/2026	**¿Se instala minikube?** (software nuevo en la máquina) + confirmación del repo público (URL) y de `KUBE_CONFIG_B64` / `DEPLOY_ENABLED` cuando exista el cluster	@user (+ @devops para ejecutarlo)	El despliegue de Kubernetes (T1.6/T4.1), el criterio C-37 y la sección de k8s del manual técnico (TD-04). El CI corre igual sin esto; el resto del stack no se toca
A-03	21/09/2026	Confirmación del **reparto de las 10 láminas** entre los 3 integrantes (la propuesta está en el backlog, sección «Entregable académico») y de quién firma/porta el DOCX final	@user + el grupo	Solo el formato de la exposición. El reparto por bloques ya está: cada integrante documenta y expone lo suyo
A-04	13/10/2026	**¿El docente acepta FastAPI como el framework del API, o exige Express.js?** La consigna del trabajo colaborativo dice «APIs REST y GraphQL con **Express.js**» y el API del proyecto es FastAPI. Se le pregunta antes de añadir un microservicio Node solo para cumplir la letra	@user (pregunta al docente)	Queda sin cubrir un fragmento de la rúbrica del ítem 20 % («APIs + BD híbridas»). Si dice que no, el plan es un microservicio Express chico que sirva el GraphQL (BFF) consumiendo el REST actual: ~1 día, sin tocar el núcleo